

PROJET *PMind*

Développement d'un moteur d'échecs *adaptatif personnalisé* par apprentissage automatique

Estimation du niveau Elo et profil comportemental
à partir de parties analysées.



PRÉSENTÉ PAR

Wafaa Berrais
Zineddine Mohammedi

ENCADREMENT

Nicolas Baskiotis - in
memoriam
Olivier Schwander

ÉTABLISSEMENT

Sorbonne Université
Master 1 MIND

Résumé

Ce mémoire étudie la modélisation d'un joueur d'échecs à partir de ses parties, dans l'objectif de préparer un moteur adaptatif et personnalisé. À partir de données collectées sur Lichess, le travail construit des features comportementales liées à la qualité des coups, à la gestion du temps, à la stabilité et aux phases de jeu. Plusieurs modèles d'apprentissage automatique sont comparés pour estimer le Elo avec un nombre limité de parties. Le mémoire propose ensuite des profils comportementaux sémantiques, corrigés par rapport au niveau prédit, afin de distinguer le niveau du joueur de sa manière de jouer. Les résultats montrent que le Elo peut être estimé de façon exploitable et que les profils apportent une information complémentaire, utile pour personnaliser l'entraînement.

Abstract

This thesis studies how to model a chess player from their games in order to support the design of an adaptive and personalized chess engine. Using data collected from Lichess, the project builds behavioral features related to move quality, time management, stability, and game phases. Several machine learning models are compared to estimate a player's Elo from a limited number of games. The thesis then introduces semantic behavioral profiles, adjusted with respect to the predicted Elo, to separate playing strength from playing style. The results show that Elo can be estimated in a practical few-game setting and that behavioral profiles provide complementary information for personalized training and adaptive engine behavior.

Mots-clés : échecs, Elo, apprentissage automatique, moteurs d'échecs, profil comportemental

Keywords : chess, Elo, machine learning, chess engines, behavioral profiling

Table des matières

Table des figures	5
Liste des figures	5
Liste des tableaux	6
Liste des tableaux	7
Liste des abréviations	8
Introduction générale	1
1 État de l’art	5
1.1 Concepts fondamentaux du jeu d’échecs	5
1.2 Les moteurs d’échecs	6
1.2.1 Les moteurs classiques	7
1.2.2 Les moteurs neuronaux	8
1.2.3 Moteurs humanisés et adaptatifs	9
2 Données et construction du dataset	11
2.1 Description des jeux de données	11
2.1.1 Source et collecte des données	11
2.1.2 Organisation et contenu des fichiers	12
2.2 Construction de jeux de données	13
2.2.1 Nettoyage et préparation des données	13
2.2.2 Agrégation hiérarchique des données	14
2.2.3 Feature Engineering	16
2.2.4 Évolution du feature engineering et de l’agrégation selon les ver- sions expérimentales	22
3 Analyse exploratoire des données	24
3.1 Analyse de la variable cible : Elo	24
3.2 Répartition par cadence et par niveau	25

3.3	Pourquoi séparer les cadences ?	26
3.4	Analyse de la qualité des coups	28
3.5	Corrélations des features avec l'Elo	33
3.6	Multicolinéarité, redondance et fuite d'information	34
3.7	Stabilité des features selon le nombre de parties	35
4	Modélisation prédictive d'elo	38
4.1	Approche générale	38
4.1.1	Protocole d'évaluation	39
4.1.2	Métriques d'évaluation	39
4.1.3	Modèles testés	40
4.2	Résultats et discussion	42
4.3	Expérience 1 – Baseline globale et effet du nombre de parties	42
4.3.1	Description	42
4.3.2	Résultats	43
4.3.3	Discussion	43
4.4	Expérience 2 – Séparation par cadence et sélection de features	44
4.4.1	Description	44
4.4.2	Résultats	44
4.4.3	Discussion	44
4.5	Expérience 3 – Ajout d'embeddings neuronaux	45
4.5.1	Description	45
4.5.2	Résultats	45
4.5.3	Discussion	46
4.6	Expérience 4 – Late aggregation et prédiction au niveau partie	46
4.6.1	Description	46
4.6.2	Résultats	47
4.6.3	Discussion	47
4.7	Synthèse des expériences de modélisation	47
5	Profil comportemental des joueurs	49
5.1	Approche générale du profiling comportemental	49
5.1.1	Construction des scores de style	50
5.1.2	Attribution des profils sémantiques	51
5.1.3	Protocole d'évaluation	52
5.2	Résultats et discussion	52
5.2.1	Résultats avec 30 parties par joueur	52

5.3	Résultats avec 10 parties par joueur	53
5.4	Discussion	55
5.5	Limites	55
5.6	Conclusion	55
	Conclusion générale	57
	Bibliographie	60

Table des figures

1.1	Exemple de position sur l'échiquier.	6
1.2	Principe de l'élagage alpha-bêta dans la recherche minimax.	7
2.1	Pipeline de collecte des données depuis l'API Lichess.	12
3.1	Distribution globale du Elo dans le dataset final.	24
3.2	Distribution du Elo par cadence.	25
3.3	Répartition des buckets Elo par cadence.	26
3.4	Distribution de <code>loss_mean_mean</code> par cadence.	27
3.5	Distribution de <code>rate_blunder_mean</code> par cadence.	27
3.6	Distribution de <code>time_mean_s_mean</code> par cadence.	27
3.7	Distribution de <code>n_moves_mean</code> par cadence.	28
3.8	Distribution de <code>cp_loss_clean</code>	28
3.9	Nombre de coups par catégorie de qualité.	29
3.10	Qualité moyenne des coups selon la phase de jeu.	29
3.11	Perte moyenne en centipions par phase et par cadence.	30
3.12	Taux de coups avec temps nul par cadence.	31
3.13	Relation entre temps utilisé et perte en centipions.	32
3.14	Variables les plus corrélées avec le Elo selon Spearman.	33
3.15	Relation entre perte moyenne et dispersion des pertes.	34
3.16	Matrice de corrélation des principales features.	35
3.17	Stabilité des features selon le nombre de parties disponibles.	36
3.18	Comparaison de la stabilité de la moyenne et de la médiane en contexte few-shot.	36
4.1	Évolution de la MAE selon le nombre de parties disponibles par joueur.	43
4.2	Importance des embeddings dans le modèle LightGBM selon la cadence.	45
5.1	Projection PCA des profils sémantiques en bullet avec 30 parties par joueur	53
5.2	Projection PCA des profils sémantiques en bullet avec 10 parties par joueur	54

Liste des tableaux

1.1	Comparaison des principales approches de moteurs d'échecs.	10
2.1	Volumes de données collectées par cadence.	12
2.2	Structure des fichiers de données.	13
2.3	Traitement des variables temporelles.	14
2.4	Niveaux d'agrégation des données.	16
2.5	Taille du dataset final par cadence.	16
2.6	Familles de variables construites lors du feature engineering.	17
2.7	Exemples de variables de qualité des coups.	17
2.8	Exemples de variables de gestion du temps.	18
2.9	Exemples de variables liées au contexte de l'erreur.	19
2.10	Exemples de variables liées aux phases de jeu.	19
2.11	Exemples de variables liées au style de jeu.	20
2.12	Exemples de variables de stabilité et cohérence.	21
2.13	Variables globales de synthèse.	21
2.14	Suffixes utilisés pour l'agrégation au niveau joueur.	22
2.15	Exemple d'agrégation d'une variable partie vers le niveau joueur.	22
3.1	Statistiques descriptives de la variable cible Elo.	25
3.2	Répartition des données par cadence.	26
3.3	Tests statistiques des différences entre cadences.	26
3.4	Répartition des coups par catégorie de qualité.	29
3.5	Qualité des coups par phase de jeu.	29
3.6	Perte moyenne en centipions par phase et par cadence.	30
3.7	Corrélations des variables de contexte avec le Elo.	30
3.8	Taux de coups avec temps utilisé nul par cadence.	31
3.9	Top corrélations de Spearman avec le Elo.	33
3.10	Exemples de paires de variables fortement corrélées.	34
3.11	Erreur d'estimation des statistiques few-shot par rapport à la référence.	36
4.1	Répartition des joueurs par cadence et ensemble.	39
4.2	Résultats principaux de l'expérience 1 sur le corpus 1 000 joueurs $\times$ 100 parties.	43

4.3	Comparaison des stratégies de sélection de features, $N = 10$ parties. . .	44
4.4	Meilleures configurations l'expérience 3 avec embeddings, pour $N = 10$ parties.	45
4.5	Résultats sans embeddings à $N = 10$ parties.	47
4.6	Résultats avec embeddings à $N = 10$ parties.	47
5.1	Familles de features utilisées pour le profiling comportemental	50
5.2	Dimensions finales de style	51
5.3	Qualité des profils sémantiques avec 30 parties par joueur	52
5.4	Exemples de profils fréquents avec 30 parties	52
5.5	Qualité des profils sémantiques avec 10 parties par joueur	53
5.6	Performance du modèle Elo utilisé comme contexte	54

Liste des abréviations

API	Application Programming Interface
CNN	Convolutional Neural Network
Elo	Système de classement d'Arpad Elo
FIDE	Fédération Internationale des Échecs
IQR	Intervalle Inter-Quartile
ML	Machine Learning (Apprentissage Automatique)
MCTS	Monte Carlo Tree Search
NNUE	Efficiently Updatable Neural Network
PGN	Portable Game Notation
RF	Random Forest
RMSE	Root Mean Squared Error

Introduction générale

Il y a quelques décennies, le grand maître Garry Kasparov affronta Deep Blue, le supercalculateur d'IBM, et perdit [17]. Ce moment symbolique marqua bien plus qu'une défaite humaine : il signifia que les machines avaient atteint, puis dépassé, l'intelligence humaine dans l'un des jeux les plus complexes jamais conçus. Depuis, les moteurs d'échecs n'ont cessé de progresser. Stockfish [6], AlphaZero [5], Leela Chess Zero [7] — des programmes capables de calculer des millions de positions par seconde, de voir à plusieurs coups à l'avance, et de jouer avec une précision que nul humain ne saurait approcher. La question n'est donc plus de savoir si une machine peut battre un champion du monde. Elle l'a fait. La vraie question, aujourd'hui, est tout autre : peut-on concevoir une machine capable de jouer comme un humain ?

Car paradoxalement, la supériorité écrasante des moteurs modernes est aussi leur principale limite dans un contexte d'apprentissage. Un joueur de 1500 points Elo confronté à Stockfish ne tire pas nécessairement un bénéfice pédagogique direct de cette rencontre : la machine joue des coups souvent difficiles à comprendre, dans des positions que le joueur ne saurait pas toujours atteindre, avec une précision qui dépasse largement le jeu humain. Affaiblir artificiellement le moteur — réduire sa profondeur de calcul ou lui imposer des erreurs aléatoires — ne suffit pas. Les travaux sur Maia l'ont montré clairement : un Stockfish bridé devient certes plus faible, mais il ne joue pas pour autant comme un humain [4]. Il joue comme une machine affaiblie, ce qui n'est pas équivalent.

L'enjeu de l'humanisation des moteurs d'échecs est précisément là. Il ne s'agit pas simplement de réduire la force d'un programme, mais de modéliser le comportement réel d'un joueur humain : ses erreurs typiques, sa gestion du temps, ses habitudes d'ouverture, sa fragilité en finale, sa réaction sous pression temporelle, sa régularité ou son instabilité d'une partie à l'autre. Humaniser un moteur, c'est lui donner une identité comportementale. Des travaux récents ont montré qu'il est possible d'apprendre des modèles personnalisés du comportement humain aux échecs [13], et même d'identifier un joueur à partir de ses seules décisions, dans une forme de stylométrie comportementale [14]. Plus récemment encore, des approches comme Maia4All ont montré qu'une telle adaptation peut être réalisée avec un nombre limité de parties [15], ouvrant la voie à des systèmes adaptatifs utilisables dès les premières interactions avec un nouvel

utilisateur.

C'est dans ce contexte que l'essor des plateformes d'échecs en ligne ouvre une opportunité importante. Des plateformes comme Lichess [16] open-source, gratuites et accessibles à des millions de joueurs dans le monde génèrent chaque jour un très grand nombre de parties, enregistrées avec leurs évaluations moteur, leurs horloges coup par coup, leurs ouvertures et leurs résultats. Ces plateformes constituent des mines de données comportementales d'une grande richesse. Pour la première fois, il devient possible d'analyser non plus seulement quelques parties de joueurs experts, mais des volumes massifs de parties de joueurs ordinaires, à différents niveaux et dans différentes cadences. Ce changement d'échelle transforme ce qu'il est possible de faire : passer de l'intuition à la mesure, de la description qualitative du style à la modélisation quantitative du comportement.

Problématique

L'objectif central de ce projet est de contribuer au développement d'un moteur d'échecs adaptatif et personnalisé. Un tel moteur doit être capable de s'ajuster au joueur qui lui fait face, non pas uniquement en force globale, mais plus finement, en tenant compte à la fois de son niveau et de sa manière de jouer. Mais avant toute adaptation, une question s'impose : comment connaître rapidement le niveau du joueur ?

Le Elo est le système de classement de référence dans le monde des échecs, introduit par le physicien Arpad Elo et adopté par la FIDE [2]. Il attribue à chaque joueur un score numérique reflétant sa force globale, mis à jour après chaque partie en fonction du résultat obtenu et du niveau de l'adversaire. Un Elo de 1200 correspond généralement à un joueur débutant, 1600 à un joueur intermédiaire, 2000 à un niveau avancé, tandis que les meilleurs joueurs mondiaux dépassent les 2800 points. C'est une mesure synthétique, lisible et largement utilisée. C'est aussi une mesure incomplète : elle indique à quel niveau joue un joueur, mais elle ne dit rien sur comment il joue.

C'est ici que se situe la problématique de ce travail. Si l'on souhaite humaniser un moteur, il faut pouvoir estimer rapidement le niveau du joueur, idéalement à partir d'un petit nombre de parties, sans attendre des semaines de jeu. Peut-on prédire le Elo d'un joueur à partir de l'analyse statistique de ses parties ? Et au-delà du niveau, deux joueurs peuvent avoir exactement le même Elo et jouer de manière très différente : l'un peut jouer vite, prendre des risques et s'effondrer sous pression ; l'autre peut jouer plus lentement, de façon plus positionnelle, avec une grande régularité mais une fragilité en finale. À niveau égal, plusieurs profils comportementaux coexistent. Un moteur réellement adaptatif doit donc non seulement estimer le niveau du joueur, mais aussi caractériser sa manière de jouer, afin de proposer une expérience d'entraînement adaptée à ses besoins.

La problématique générale de ce mémoire peut alors se formuler ainsi :

Comment modéliser un joueur d'échecs à partir de ses parties afin d'estimer son niveau et de caractériser son profil comportemental, dans la perspective d'un moteur d'échecs adaptatif et humanisé ?

Objectifs et approche

Pour répondre à cette problématique, ce projet adopte une approche fondée sur le feature engineering et les modèles d'apprentissage supervisé tabulaires. Plutôt que de recourir uniquement à des architectures neuronales profondes de bout en bout puissantes mais souvent opaques, gourmandes en données et difficiles à interpréter on construit d'abord une représentation manuelle et lisible du joueur : ses pertes moyennes en centipions, ses taux d'erreurs par phase de jeu, sa gestion du temps, sa fragilité sous pression temporelle, sa stabilité inter-parties, ainsi que son comportement en position gagnante, équilibrée ou défavorable. Ces variables forment un profil statistique structuré, où chaque dimension correspond à une réalité comportementale concrète et interprétable.

Cette approche présente plusieurs avantages. Elle est interprétable : chaque variable a une signification échiquéenne claire, ce qui permet d'expliquer les prédictions et de les relier à des comportements réels. Elle est économe en données : elle vise à produire des estimations exploitables à partir d'un nombre limité de parties. Elle est rapide : les modèles de gradient boosting sont bien adaptés aux données tabulaires et s'entraînent efficacement. Enfin, elle est flexible : les mêmes variables comportementales qui servent à prédire le Elo peuvent ensuite être réutilisées pour construire des profils de style. Le Elo prédit situe le joueur sur une échelle de force ; le profil comportemental décrit sa manière de jouer par rapport à des joueurs de niveau similaire.

Les objectifs concrets de ce mémoire sont les suivants :

- collecter et construire un dataset de parties issues de Lichess, séparées par cadence ;
- extraire des features comportementales structurées au niveau joueur ;
- analyser les relations entre ces variables, le niveau Elo et la cadence de jeu ;
- comparer plusieurs stratégies de modélisation pour prédire le Elo avec peu de parties ;
- construire, au-delà du niveau, des scores de style corrigés par rapport au Elo prédit ;
- proposer une typologie sémantique de profils comportementaux interprétables.

Ce rapport est organisé en cinq chapitres. Le chapitre 1 présente l'état de l'art sur les moteurs d'échecs classiques et neuronaux, ainsi que les travaux existants sur l'humanisation et la personnalisation du comportement. Le chapitre 2 décrit la collecte

des données depuis Lichess et la construction du dataset, avec un accent particulier sur la logique de feature engineering. Le chapitre 3 est consacré à l'analyse exploratoire : distribution du Elo, différences entre cadences, corrélations des features et stabilité en contexte few-shot. Le chapitre 4 présente les expériences de modélisation du Elo, en comparant plusieurs approches progressives : baseline globale, séparation par cadence, embeddings neuronaux et prédiction au niveau partie. Enfin, le chapitre 5 prolonge la démarche vers le profil comportemental : à niveau comparable, comment identifier et décrire les différents styles de joueurs de manière interprétable et stable ?

Chapitre 1

État de l’art

Trois niveaux de connaissances sont nécessaires pour situer ce travail : les règles et notions propres aux échecs, le fonctionnement des moteurs d’analyse, et les approches existantes pour modéliser le joueur humain. Chacun de ces niveaux est présenté successivement dans cette section.

1.1 Concepts fondamentaux du jeu d’échecs

Du point de vue de l’intelligence artificielle et de la théorie des jeux, les échecs peuvent être considérés comme un jeu de stratégie à deux joueurs, déterministe, à information parfaite et sans hasard : chaque joueur connaît l’état complet de l’échiquier, et chaque décision dépend uniquement de la position courante et des coups précédents [1]. Cette structure explique pourquoi les échecs constituent historiquement un domaine de référence pour étudier la recherche de coups, l’évaluation de positions et la prise de décision en intelligence artificielle [8].

Les échecs opposent deux joueurs, les Blancs et les Noirs, sur un échiquier de 64 cases. Chaque joueur commence la partie avec seize pièces : un roi, une dame, deux tours, deux fous, deux cavaliers et huit pions. Les joueurs jouent chacun leur tour, et chaque coup légal transforme la position courante en une nouvelle position. L’objectif principal est de mettre le roi adverse en situation d’échec et mat. Selon les règles officielles de la FIDE, un joueur gagne lorsqu’il donne échec et mat au roi adverse par un coup légal ; une partie peut aussi se terminer par une nulle dans plusieurs situations, par exemple lorsqu’aucun joueur ne dispose du matériel nécessaire pour mater [1].

Plusieurs notions sont importantes pour décrire une partie d’échecs [3] :

- une **position** désigne l’état du plateau à un moment donné ;
- un **coup** correspond au déplacement légal d’une pièce ;
- une **partie** est une suite de positions et de coups joués alternativement par les deux joueurs.



FIGURE 1.1 – Exemple de position sur l'échiquier.

Une partie est généralement divisée en trois grandes phases : l'ouverture, qui correspond aux premiers coups ; le milieu de jeu, où les plans stratégiques et tactiques se développent ; et la finale, lorsque le nombre de pièces diminue et que la précision technique devient plus importante [3].

Les parties peuvent aussi être jouées avec différentes cadences, c'est-à-dire différents temps de réflexion. Les formats rapides comme le bullet ou le blitz laissent peu de temps à chaque joueur, tandis que les formats rapid ou classique offrent davantage de temps. Cette différence influence fortement le comportement des joueurs : un même joueur peut prendre plus de risques, jouer plus vite ou commettre davantage d'erreurs selon la cadence [3].

Enfin, chaque joueur peut être associé à un niveau, souvent représenté par un score de type Elo ou Glicko selon la plateforme. Ce score résume globalement la force du joueur à partir de ses résultats passés, mais il ne décrit pas directement sa manière de jouer [3]. Pour analyser plus finement une partie, il devient alors nécessaire d'utiliser des outils capables d'évaluer les positions et les coups joués, ce qui mène naturellement à l'étude des moteurs d'échecs.

1.2 Les moteurs d'échecs

Un moteur d'échecs est un programme qui analyse une position donnée, explore les coups possibles et sélectionne le coup considéré comme le plus favorable selon une méthode de recherche et/ou d'évaluation. Dans la littérature en programmation d'échecs, la recherche consiste à regarder à l'avance plusieurs séquences de coups possibles puis à évaluer les positions obtenues [8]. Les moteurs modernes peuvent être regroupés en

deux grandes familles : les moteurs classiques, fondés principalement sur la recherche dans l'arbre de jeu et une fonction d'évaluation, et les moteurs neuronaux, qui utilisent des réseaux de neurones pour évaluer les positions ou guider la recherche.

1.2.1 Les moteurs classiques

Les moteurs classiques, comme Stockfish, combinent généralement deux composantes principales : une fonction de recherche et une fonction d'évaluation [6].

Recherche. Le moteur explore un arbre de coups possibles à partir de la position courante. Le principe est proche du minimax : le moteur choisit le coup qui maximise son avantage en supposant que l'adversaire cherchera lui aussi à jouer le meilleur coup. Pour éviter d'explorer inutilement tout l'arbre, les moteurs classiques utilisent des techniques d'élagage comme l'alpha-beta pruning, qui supprime certaines branches qui ne peuvent pas améliorer la décision finale [8].

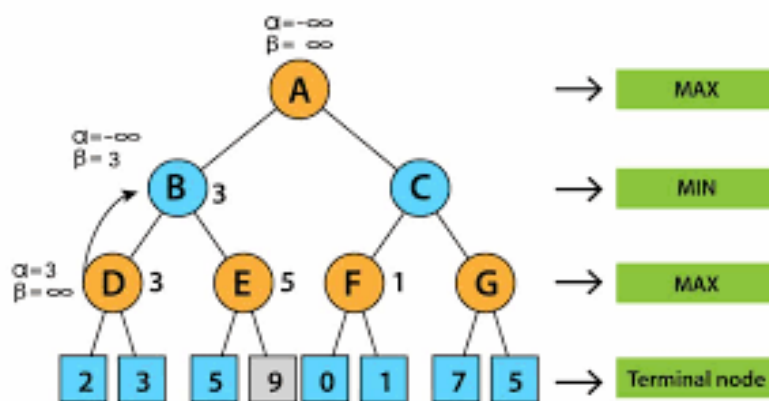


FIGURE 1.2 – Principe de l'élagage alpha-bêta dans la recherche minimax.

Évaluation. Lorsque le moteur ne peut pas explorer jusqu'à la fin de la partie, il utilise une fonction d'évaluation pour attribuer une valeur numérique à une position. Cette fonction estime la valeur relative de la position, c'est-à-dire les chances de gain, de nulle ou de défaite [9]. Dans les moteurs classiques, cette évaluation peut être construite à partir de concepts échiquiers tels que le matériel, la sécurité du roi, la structure de pions ou l'activité des pièces.

Une forme simplifiée d'évaluation peut être vue comme une somme pondérée de caractéristiques de position :

$$\text{Eval}(\text{position}) = w_1 f_1 + w_2 f_2 + \dots + w_n f_n \quad (1.1)$$

où f_i représente une caractéristique de la position et w_i son poids. Dans les moteurs modernes, cette idée est beaucoup plus complexe et fortement optimisée.

Les scores sont souvent exprimés en centipawns : 100 centipawns correspondent approximativement à la valeur d'un pion. Stockfish indique que les évaluations ont traditionnellement été mesurées en pions ou en centipawns, avec 1 pion = 100 centipawns [10].

Évolution récente : NNUE. Même dans les moteurs dits classiques, l'évaluation a évolué. Stockfish utilise aujourd'hui une évaluation neuronale de type NNUE. NNUE est une architecture de réseau neuronal conçue pour remplacer ou améliorer l'évaluation dans des moteurs à recherche alpha-beta tout en restant très rapide sur CPU [11]. Stockfish explique que l'évaluation classique calcule une valeur à partir de concepts échiquiers conçus et optimisés par des experts, tandis que l'évaluation NNUE calcule cette valeur avec un réseau neuronal entraîné sur de nombreuses positions [12].

Ces moteurs sont très puissants pour trouver de bons coups, mais leur objectif principal reste la performance objective. Le problème est qu'un moteur peut être très fort, ou même être affaibli artificiellement, sans reproduire la manière dont un humain joue. Les travaux sur Maia montrent justement que limiter la profondeur de Stockfish ne suffit pas à obtenir un comportement humain ciblé par niveau [4].

1.2.2 Les moteurs neuronaux

Les moteurs neuronaux utilisent des réseaux de neurones pour apprendre à évaluer les positions et à sélectionner les coups prometteurs. Contrairement aux moteurs classiques, ils s'appuient moins sur une fonction d'évaluation entièrement écrite à la main et davantage sur des représentations apprises à partir de données ou par auto-jeu. AlphaZero est l'un des exemples les plus connus : il apprend en jouant contre lui-même et combine réseau neuronal et Monte Carlo Tree Search [5]. Leela Chess Zero reprend cette approche dans un moteur open-source inspiré d'AlphaZero [7].

Recherche. La recherche des moteurs neuronaux est généralement guidée par le réseau neuronal. Au lieu d'explorer massivement l'arbre avec alpha-beta, le moteur concentre l'exploration sur les coups jugés les plus prometteurs. Dans AlphaZero et Leela Chess Zero, cette recherche est basée sur Monte Carlo Tree Search, qui explore progressivement les variantes en utilisant les prédictions du réseau [5, 7].

Évaluation. L'évaluation d'une position est produite par le réseau neuronal. Celui-ci peut estimer :

- la valeur de la position, c'est-à-dire les chances de gain ou de perte ;
- la probabilité des coups intéressants à jouer depuis cette position.

Cette évaluation est apprise à partir de nombreuses parties ou par auto-jeu, au lieu d'être uniquement construite à partir de règles explicites comme le matériel ou la structure de pions [5].

Les moteurs neuronaux ont montré qu'il est possible d'atteindre un très haut niveau de jeu avec des représentations apprises. Cependant, leur objectif reste généralement de trouver le meilleur coup possible. Ils ne sont donc pas automatiquement adaptés pour reproduire les décisions, les erreurs ou les habitudes d'un joueur humain. Cette limite mène à la question des moteurs humanisés et adaptatifs.

1.2.3 Moteurs humanisés et adaptatifs

Les moteurs classiques et neuronaux ont atteint un niveau très élevé, parfois largement supérieur au niveau humain. Cependant, un moteur très fort n'est pas automatiquement un bon partenaire d'entraînement. Dans un contexte d'apprentissage, l'objectif n'est pas seulement de produire le meilleur coup, mais aussi de proposer une opposition compréhensible, progressive et adaptée au joueur.

Limite de l'adaptation par la force. Une première manière d'adapter un moteur consiste à réduire sa force : limiter la profondeur de recherche, diminuer le temps de calcul ou introduire des erreurs artificielles. Cette approche permet de créer un adversaire moins fort, mais elle ne garantit pas un comportement humain. Les travaux sur Maia montrent par exemple que les versions de Stockfish limitées en profondeur ne reproduisent pas forcément les coups typiques de joueurs faibles ou intermédiaires : elles deviennent plus faibles, mais pas nécessairement plus humaines [4].

Modéliser les coups humains. Maia est une approche importante dans les moteurs humanisés, car son objectif n'est pas de jouer le meilleur coup possible, mais de prédire le coup qu'un humain jouerait réellement. Techniquement, Maia est une version adaptée d'AlphaZero, mais contrairement à AlphaZero qui apprend par auto-jeu pour maximiser la performance, Maia est entraînée sur des parties humaines issues de Lichess. Le modèle apprend donc à imiter les décisions humaines plutôt qu'à produire uniquement des coups optimaux. Les auteurs entraînent plusieurs versions de Maia correspondant à différents niveaux Elo, par exemple de 1100 à 1900, afin que le modèle soit capable de reproduire les choix typiques de joueurs d'un niveau donné. Cette approche permet à Maia de mieux prédire les coups humains que des moteurs classiques ou neuronaux optimisés pour la force brute, comme Stockfish ou Leela Chess Zero [4].

Cette idée change la définition de l'adaptation : il ne s'agit plus seulement de régler une force globale, mais de rapprocher le comportement du moteur de celui des humains. Pour cela, plusieurs dimensions du joueur deviennent importantes : son niveau, ses

erreurs, sa gestion du temps, sa stabilité, ses préférences de jeu et ses performances selon les phases de la partie.

Personnalisation et style individuel. L'adaptation devient plus fine lorsqu'on cherche à modéliser un joueur particulier. Des travaux sur les modèles personnalisés montrent qu'il est possible d'améliorer la prédiction des coups d'un joueur donné en spécialisant un modèle sur ses propres parties. Ces modèles capturent des signatures individuelles du comportement, au point de permettre une forme de stylométrie : identifier un joueur à partir de ses décisions [13].

D'autres travaux sur la stylométrie comportementale confirment que les parties d'échecs contiennent des marqueurs de style individuel. Ils montrent qu'un joueur peut être identifié à partir de ses coups, même lorsque l'on évite de se baser uniquement sur les ouvertures, ce qui suggère que le style ne se réduit pas au niveau Elo [14].

Adaptation avec peu de données. Un enjeu important reste la quantité de données nécessaire. Les premières approches de personnalisation nécessitent souvent un grand nombre de parties par joueur, ce qui limite leur usage pour un nouvel utilisateur. Maia4All propose une approche plus efficace en utilisant des prototypes de joueurs et des embeddings pour adapter le modèle à un joueur avec beaucoup moins de données. L'article rapporte une adaptation du comportement individuel avec seulement 20 parties, contre plusieurs milliers dans des approches précédentes [15].

Ces travaux montrent que l'adaptation d'un moteur d'échecs peut être abordée à plusieurs niveaux : adapter la force, imiter les décisions humaines, personnaliser le comportement et tenir compte du style. Dans notre cas, cela motive une première étape centrée sur l'estimation rapide du niveau du joueur, avant d'aller vers l'identification de profils comportementaux plus fins.

TABLE 1.1 – Comparaison des principales approches de moteurs d'échecs.

Moteur	Type	Évaluation	Recherche
Stockfish	Classique/NNUE	Fonction + réseau NNUE	Alpha-bêta
AlphaZero	Neuronal	Réseau neuronal	MCTS
Leela Chess Zero	Neuronal	Réseau neuronal	MCTS
Maia	Neuronal	Prédiction du coup humain	Recherche guidée

Chapitre 2

Données et construction du dataset

Cette section décrit les données utilisées dans le projet, depuis leur collecte jusqu'à la construction du dataset final. Elle présente les sources, l'organisation des fichiers, les étapes de nettoyage, puis la logique de feature engineering qui transforme les coups bruts en un profil statistique exploitable par joueur et par cadence.

2.1 Description des jeux de données

2.1.1 Source et collecte des données

Les données utilisées dans ce projet proviennent de Lichess [16], une plateforme d'échecs en ligne qui met à disposition des parties, des métadonnées de joueurs et des évaluations moteur. La collecte a été réalisée à l'aide d'un script Python utilisant l'API Lichess. Le pipeline interroge d'abord des sources de candidats joueurs, comme les classements et les résultats de tournois, puis télécharge les parties analysées des joueurs retenus.

Dans une première étape du projet, une collecte plus globale a été réalisée, avec des corpus de type $1000 \text{ joueurs} \times 100 \text{ parties}$, sans séparation stricte par cadence. Cette première version a permis de tester la faisabilité de la prédiction du Elo à partir des parties. Par la suite, afin d'analyser plus finement les différences de comportement entre bullet, blitz et rapid, la collecte a été restructurée en corpus séparés par cadence.

Pour chaque joueur candidat, plusieurs critères sont imposés lors de la récupération des parties : les parties doivent être classées, déjà analysées, contenir des évaluations moteur, inclure les informations d'horloge, fournir les informations d'ouverture et contenir la représentation PGN dans le JSON. La cadence est filtrée directement au niveau de l'API afin de séparer les corpus bullet, blitz et rapid.

L'objectif de cette seconde collecte était de construire, pour chaque cadence, un corpus équilibré de type $700 \text{ joueurs} \times 30 \text{ parties}$. Les joueurs sont répartis en cinq intervalles Elo : $[1000, 1400[$, $[1400, 1600[$, $[1600, 1800[$, $[1800, 2000[$ et $[2000, 2200]$, avec

une cible de 140 joueurs par intervalle. Un joueur n'est conservé que si le pipeline parvient à garder exactement 30 parties valides dans la cadence ciblée.



FIGURE 2.1 – Pipeline de collecte des données depuis l'API Lichess.

À la fin de la collecte, les volumes obtenus sont les suivants :

TABLE 2.1 – Volumes de données collectées par cadence.

Cadence	Joueurs	Parties	Demi-coups	Elo moyen
Bullet	700	21 000	1 282 007	1681.6
Blitz	700	21 000	1 414 049	1672.4
Rapid	486	14 580	984 913	1582.7
Total	1 886	56 580	3 680 969	—

Les corpus bullet et blitz respectent la cible initiale. Le corpus rapid est plus réduit, car plusieurs joueurs candidats ne disposaient pas de 30 parties classées, analysées et exploitables dans la cadence ciblée.

2.1.2 Organisation et contenu des fichiers

Les données sont organisées par cadence. Chaque dossier de cadence contient trois fichiers principaux : `players.csv`, `games.csv` et `moves.csv`. Ces fichiers correspondent à trois granularités différentes : joueur, partie et coup. Le fichier `players.csv` contient les informations agrégées au niveau joueur. Il inclut notamment le nom du joueur, son Elo, son intervalle Elo, le nombre de parties collectées et quelques indicateurs globaux comme la perte moyenne en centipions, les taux d'erreurs et le taux de victoire.

Le fichier `games.csv` contient une ligne par partie. Il permet de relier chaque partie à son contexte : cadence, contrôle de temps, date, ouverture, joueur cible, adversaire, résultat et couleur jouée.

Le fichier `moves.csv` est le plus détaillé. Chaque ligne correspond à un demi-coup, c'est-à-dire un coup joué par l'un des deux joueurs. Il contient les informations nécessaires pour analyser les décisions du joueur : position avant le coup, coup joué, évaluation moteur avant/après, meilleur coup moteur, perte en centipions, phase de jeu, temps restant et temps utilisé.

Dans la formulation supervisée du problème, l'entrée du modèle est constituée des variables construites à partir des parties et des coups, tandis que la sortie à prédire est le Elo du joueur. La variable `elo_bucket` est conservée pour les analyses par tranche de niveau.

TABLE 2.2 – Structure des fichiers de données.

Fichier	Granularité	Contenu principal	Rôle dans le projet
<code>players.csv</code>	Joueur	Identifiant du joueur, Elo, bucket Elo, nombre de parties et statistiques globales de qualité.	Définit le profil agrégé du joueur et fournit la cible principale de prédiction : le Elo.
<code>games.csv</code>	Partie	Identifiant de partie, date, cadence, ouverture, joueurs, Elo, résultat et couleur du joueur cible.	Décrit le contexte de chaque partie et relie les observations au joueur étudié.
<code>moves.csv</code>	Demi-coup	Position, coup joué, évaluation moteur, perte en centipions, temps et indicateurs tactiques.	Fournit les informations fines utilisées pour construire les features comportementales.

2.2 Construction de jeux de données

2.2.1 Nettoyage et préparation des données

Avant la construction du dataset final, un prétraitement est appliqué afin de rendre les données cohérentes et exploitables. Le pipeline conserve principalement les coups joués par le joueur cible, puis filtre les parties non conformes : mauvaise cadence, variante non standard, absence d'analyse moteur ou informations incomplètes.

Les évaluations moteur sont nettoyées pour limiter l'effet des valeurs extrêmes, notamment dans les positions de mat ou très déséquilibrées. La perte en centipions est représentée par `cp_loss_clean`, calculée à partir de la valeur absolue de `cp_loss` et bornée afin d'obtenir une mesure plus stable.

Un traitement spécifique est aussi appliqué aux variables temporelles :

TABLE 2.3 – Traitement des variables temporelles.

Variable	Problème	Traitement
time_spent_cs	Valeurs nulles	Remplacées par 0, interprétées comme des coups très rapides ou des premoves
clock_before_cs	Quelques valeurs manquantes	Remplacées par la médiane de la colonne

Enfin, les variables temporelles sont converties en secondes lorsque nécessaire, puis utilisées pour construire les indicateurs liés au rythme de jeu et à la pression temporelle.

2.2.2 Agrégation hiérarchique des données

Comme la variable cible est le Elo du joueur, les données doivent être transformées progressivement depuis le niveau coup vers le niveau joueur. La construction suit donc une logique hiérarchique :

Algorithm 1 Construction hiérarchique du dataset joueur-cadence

```

1: Input :  $\mathcal{M}$  table des coups,  $\mathcal{G}$  table des parties,  $\mathcal{P}$  table des joueurs
2: Output : Dataset final  $\mathcal{D}_{player}$  au niveau joueur-cadence

                                     ▷ Prétraitement
3:  $\mathcal{M}^* \leftarrow \text{PRETRAITERCOUPS}(\mathcal{M})$ 
4:  $\mathcal{G}^* \leftarrow \text{PRETRAITERPARTIES}(\mathcal{G})$ 

                                     ▷ Extraction des features au niveau coup
5: for chaque coup  $m \in \mathcal{M}^*$  do
6:    $\phi_{move}(m) \leftarrow \text{EXTRAIREFEATURESCOUP}(m)$ 
7: end for

                                     ▷ Agrégation coups  $\rightarrow$  parties
8:  $\mathcal{D}_{game} \leftarrow \emptyset$ 
9: for chaque partie  $g \in \mathcal{G}^*$  do
10:    $\mathcal{M}_g^* \leftarrow \{m \in \mathcal{M}^* \mid m.\text{game\_id} = g.\text{game\_id}\}$ 
11:    $\mathbf{x}_g \leftarrow \text{AGREGERCOUPSVERSPARTIE}(\mathcal{M}_g^*)$ 
12:    $\mathbf{x}_g \leftarrow \text{AJOUTERMETADONNEES}(\mathbf{x}_g, g)$ 
13:    $\mathcal{D}_{game} \leftarrow \mathcal{D}_{game} \cup \{(g.\text{target\_name}, g.\text{speed}, \mathbf{x}_g)\}$ 
14: end for

                                     ▷ Agrégation parties  $\rightarrow$  joueur-cadence
15:  $\mathcal{D}_{player} \leftarrow \emptyset$ 
16: for chaque joueur  $j$  et cadence  $c$  do
17:    $\mathcal{G}_{j,c} \leftarrow \{g \in \mathcal{D}_{game} \mid g.\text{target\_name} = j \wedge g.\text{speed} = c\}$ 
18:   if  $|\mathcal{G}_{j,c}| > 0$  then
19:      $\mathbf{x}_{j,c} \leftarrow \text{AGREGERPARTIESVERSJOUEUR}(\mathcal{G}_{j,c})$ 
20:      $y_{j,c} \leftarrow \text{EXTRAIREELO}(j, c, \mathcal{P})$ 
21:      $\mathcal{D}_{player} \leftarrow \mathcal{D}_{player} \cup \{(\mathbf{x}_{j,c}, y_{j,c})\}$ 
22:   end if
23: end for

                                     ▷ Finalisation
24:  $\mathcal{D}_{player} \leftarrow \text{SUPPRIMERFUITES}(\mathcal{D}_{player})$ 
25:  $\mathcal{D}_{player} \leftarrow \text{EXPORTERDATASET}(\mathcal{D}_{player})$ 
26: return  $\mathcal{D}_{player}$ 

```

Dans une première étape, les coups sont regroupés par partie. On obtient alors une table *game-level* où chaque ligne résume une partie du point de vue du joueur cible. Cette table contient des indicateurs de qualité, de temps, d'erreurs, de contexte positionnel, de phase de jeu et de style tactique.

Dans une deuxième étape, les parties sont regroupées par joueur et par cadence. Un même joueur peut donc être représenté par plusieurs profils s'il apparaît dans plusieurs cadences. Ce choix permet de modéliser séparément les comportements en bullet, blitz et rapid.

TABLE 2.4 – Niveaux d’agrégation des données.

Niveau	Description	Exemples de variables
Coup	Décision individuelle dans une position donnée	cp_loss_clean, is_blunder, time_spent_s, phase
Partie	Résumé du comportement du joueur dans une partie	loss_mean, rate_blunder, time_mean_s, rate_capture
Joueur	Profil global construit à partir de plusieurs parties	loss_mean_mean, rate_error_iqr, time_mean_s_median, win_rate

Le dataset final contient 1 886 profils joueur-cadence $\times$ 219 colonnes :

TABLE 2.5 – Taille du dataset final par cadence.

Cadence	Profils joueur-cadence	Colonnes
Bullet	700	219
Blitz	700	219
Rapid	486	219

Les variables finales sont principalement construites avec des statistiques comme la moyenne, la médiane, l’écart-type et l’IQR. La moyenne décrit le comportement moyen du joueur, tandis que l’écart-type ou l’IQR renseignent sur sa stabilité d’une partie à l’autre.

2.2.3 Feature Engineering

Le feature engineering constitue une étape centrale de notre approche. L’objectif n’est pas seulement d’utiliser les statistiques déjà présentes dans les fichiers collectés, mais de construire une représentation plus complète du joueur à partir de ses coups et de ses parties. Cette représentation doit permettre de décrire à la fois son niveau moyen, sa régularité, sa gestion du temps, ses erreurs typiques et certains éléments de son style de jeu.

Les variables ont été construites selon une logique hiérarchique. Les informations fines sont d’abord extraites au niveau des coups, puis agrégées au niveau des parties, avant d’être résumées au niveau joueur. Cette approche permet de transformer une séquence de décisions individuelles en un profil statistique exploitable pour la prédiction du Elo.

L’idée générale est la suivante :

coups du joueur $\rightarrow$ résumé par partie $\rightarrow$ résumé par joueur $\rightarrow$ prédiction du Elo

Le feature engineering a été structuré autour de plusieurs familles de variables complémentaires :

TABLE 2.6 – Familles de variables construites lors du feature engineering.

Famille de variables	Objectif principal
Qualité des coups	Mesurer la précision moyenne du joueur et la gravité de ses erreurs
Gestion du temps	Décrire le rythme de jeu et la pression temporelle
Contexte de l'erreur	Identifier dans quelles situations le joueur se trompe
Phases de jeu	Comparer le comportement en ouverture, milieu de jeu et finale
Style de jeu	Capturer certains marqueurs comportementaux comme les captures, les échecs ou la longueur des parties
Stabilité	Mesurer la régularité du joueur entre ses parties
Variables globales	Résumer le volume de données, le résultat moyen et la diversité des ouvertures

Cette structuration permet d'aller au-delà d'une simple moyenne globale de qualité. Deux joueurs ayant un Elo proche peuvent avoir des comportements très différents : l'un peut être rapide mais instable, un autre plus lent mais régulier, ou encore un joueur peut être solide en ouverture mais fragile en finale.

a) Qualité des coups

Le premier groupe de variables décrit la qualité brute des coups joués. Ces variables s'appuient principalement sur la perte en centipions associée aux coups du joueur. Elles permettent de mesurer la précision moyenne, la dispersion des erreurs et la fréquence des fautes importantes.

TABLE 2.7 – Exemples de variables de qualité des coups.

Variable	Description	Utilité
loss_mean	Perte moyenne en centipions dans une partie	Mesure la qualité globale des décisions
loss_median	Médiane de la perte	Donne une mesure robuste, moins sensible aux coups extrêmes
loss_p90	90e percentile de la perte	Capture les mauvaises décisions fréquentes sans se limiter au pire coup
loss_max	Perte maximale observée	Mesure la plus grosse erreur de la partie
rate_blunder	Proportion de blunders	Mesure la fréquence des grosses erreurs
rate_error	Proportion globale de coups erronés	Résume la fréquence générale des erreurs

b) Gestion du temps

La gestion du temps est une dimension importante, en particulier dans les cadences rapides comme bullet et blitz. Ce bloc ne cherche pas seulement à savoir si un joueur joue vite ou lentement, mais aussi s'il parvient à conserver une bonne qualité de jeu sous contrainte temporelle.

TABLE 2.8 – Exemples de variables de gestion du temps.

Variable	Description	Utilité
time_mean_s	Temps moyen passé par coup	Mesure le rythme global de réflexion
time_median_s	Temps médian par coup	Représente le temps typique de décision
time_std_s	Variabilité du temps par coup	Mesure l'irrégularité de gestion du temps
clock_mean_s	Temps moyen restant avant les coups	Indique le niveau global de pression temporelle
time_mean_on_blunder	Temps moyen utilisé sur les blunders	Permet de voir si les grosses erreurs sont liées à la précipitation
blunder_low_time_rate	Proportion de blunders sous faible temps	Mesure la fragilité sous pression temporelle
good_fast_move_rate	Proportion de bons coups joués rapidement	Capture la capacité à décider vite sans perdre en qualité

Ces variables sont particulièrement utiles pour distinguer des profils de joueurs. Par exemple, deux joueurs peuvent avoir une précision moyenne proche, mais l'un peut maintenir cette précision sous faible temps alors que l'autre s'effondre en zeitnot.

c) Contexte de l'erreur

Toutes les erreurs n'ont pas la même signification. Une erreur dans une position équilibrée ou gagnante peut être plus informative qu'une erreur dans une position déjà perdante. Pour cette raison, certaines variables croisent la qualité des coups avec le contexte positionnel.

Ce bloc permet de mieux comprendre dans quelles situations un joueur perd en qualité. Il est donc utile non seulement pour prédire le niveau, mais aussi pour préparer une analyse de profil comportemental.

TABLE 2.9 – Exemples de variables liées au contexte de l’erreur.

Variable	Description	Utilité
loss_when_winning	Perte moyenne en position gagnante	Mesure la capacité à conserver un avantage
loss_when_equal	Perte moyenne en position équilibrée	Analyse la qualité dans les positions les plus révélatrices
loss_when_losing	Perte moyenne en position perdante	Mesure le comportement en situation difficile
rate_blunder_when_equal	Taux de blunders en position équilibrée	Identifie les grosses erreurs dans les moments critiques
rate_blunder_when_winning	Taux de blunders en position gagnante	Mesure les erreurs de conversion
rate_blunder_when_losing	Taux de blunders en position perdante	Analyse la dégradation du jeu en situation défavorable

d) Phases de jeu

Les performances d’un joueur peuvent varier selon la phase de la partie. Certains joueurs ont une bonne préparation d’ouverture mais se dégradent ensuite ; d’autres sont plus solides en finale. Pour cette raison, plusieurs variables sont calculées séparément en ouverture, milieu de jeu et finale.

TABLE 2.10 – Exemples de variables liées aux phases de jeu.

Variable	Description
loss_opening_mean	Perte moyenne en ouverture
loss_middlegame_mean	Perte moyenne en milieu de jeu
loss_endgame_mean	Perte moyenne en finale
rate_blunder_opening	Taux de blunders en ouverture
rate_blunder_middlegame	Taux de blunders au milieu de jeu
time_endgame_mean_s	Temps moyen utilisé en finale
opening_vs_endgame_loss_gap	Différence de qualité entre finale et ouverture

Ces variables permettent de détecter des profils comme : joueur solide en ouverture, joueur fragile en finale, joueur qui perd en précision au fil de la partie, etc.

e) Style de jeu

Le style de jeu est plus difficile à mesurer directement que la qualité des coups. Les variables utilisées ici ne décrivent donc pas tout le style d’un joueur, mais elles donnent

des indices sur certaines tendances : activité tactique, agressivité, simplification, longueur des parties ou diversité des ouvertures.

TABLE 2.11 – Exemples de variables liées au style de jeu.

Variable	Description	Interprétation possible
rate_capture	Proportion de coups qui sont des captures	Indice d'activité tactique ou de jeu concret
rate_check	Proportion de coups donnant échec	Indice d'agressivité tactique
rate_castle	Fréquence de roque	Indice de sécurité du roi et de discipline positionnelle
rate_promotion	Proportion de promotions	Indice lié aux finales avancées
n_moves	Nombre de coups du joueur dans la partie	Indique la longueur des parties
ply_count	Nombre total de demi-coups	Décrit la durée globale de la partie
eco_diversity	Nombre de familles d'ouvertures différentes jouées	Mesure la diversité du répertoire d'ouverture

Ces variables ne suffisent pas seules à définir un style complet, mais elles contribuent à distinguer des tendances comportementales lorsqu'elles sont combinées avec les variables de qualité, de temps et de stabilité.

f) Stabilité et cohérence

La stabilité décrit principalement la régularité du joueur entre ses parties. Un joueur peut avoir une performance moyenne correcte mais être très irrégulier : certaines parties sont très propres, d'autres contiennent beaucoup d'erreurs. Cette variabilité est importante pour distinguer les joueurs fiables des joueurs instables.

Certaines variables décrivent aussi l'évolution au sein d'une partie, par exemple `loss_drift_end_vs_start`, qui mesure la différence de qualité entre la fin et le début de partie. Cela permet d'observer si le joueur se dégrade au fil de la partie.

g) Variables globales de synthèse

Enfin, quelques variables globales sont conservées pour résumer le profil général du joueur et le contexte des données disponibles.

Le Elo est conservé comme variable cible et n'est pas utilisé comme feature explicative dans les modèles de prédiction.

Agrégation au niveau joueur. Les variables calculées au niveau partie sont ensuite agrégées au niveau joueur. Selon les versions du pipeline, plusieurs statistiques sont

TABLE 2.12 – Exemples de variables de stabilité et cohérence.

Variable	Description	Utilité
<code>between_game_loss_variance</code>	Variance de la perte moyenne entre les parties	Mesure l'irrégularité globale de la qualité
<code>between_game_error_variance</code>	Variance du taux d'erreur entre les parties	Mesure l'instabilité du volume d'erreurs
<code>loss_mean_cv</code>	Coefficient de variation de la perte moyenne	Mesure la dispersion relative de la qualité
<code>rate_error_cv</code>	Coefficient de variation du taux d'erreur	Mesure la dispersion relative des erreurs
<code>prop_games_with_blunder</code>	Proportion de parties avec au moins un blunder	Mesure la fréquence des effondrements graves
<code>prop_games_with_mistake</code>	Proportion de parties avec au moins une mistake	Mesure la fréquence des erreurs significatives

TABLE 2.13 – Variables globales de synthèse.

Variable	Description	Utilité
<code>n_games</code>	Nombre de parties disponibles pour le joueur	Indique la robustesse statistique du profil
<code>win_rate</code>	Score moyen obtenu par le joueur	Résume sa performance globale dans l'échantillon
<code>eco_diversity</code>	Diversité des familles d'ouvertures jouées	Décrit la variété du répertoire
<code>dominant_speed</code>	Cadence dominante du profil	Indique le contexte principal de jeu
<code>elo</code>	Elo du joueur	Variable cible de la tâche supervisée
<code>elo_bucket</code>	Intervalle Elo du joueur	Utilisé pour les analyses par tranche de niveau

utilisées :

TABLE 2.14 – Suffixes utilisés pour l’agrégation au niveau joueur.

Suffixe	Signification	Rôle
<code>_mean</code>	Moyenne de la statistique sur les parties du joueur	Décrit le comportement moyen
<code>_median</code>	Médiane sur les parties	Donne une mesure robuste aux valeurs extrêmes
<code>_std</code>	Écart-type entre les parties	Mesure la variabilité
<code>_iqr</code>	Intervalle interquartile	Mesure la dispersion robuste

Par exemple, une variable comme `loss_mean` au niveau partie peut produire plusieurs variables au niveau joueur :

TABLE 2.15 – Exemple d’agrégation d’une variable partie vers le niveau joueur.

Variable joueur-level	Signification
<code>loss_mean_mean</code>	Moyenne des pertes moyennes sur les parties du joueur
<code>loss_mean_median</code>	Médiane des pertes moyennes
<code>loss_mean_std</code>	Variabilité de la perte moyenne entre les parties
<code>loss_mean_iqr</code>	Dispersion robuste de la perte moyenne entre les parties

Cette étape permet de représenter un joueur non seulement par son comportement moyen, mais aussi par sa régularité et sa variabilité d’une partie à l’autre.

2.2.4 Évolution du feature engineering et de l’agrégation selon les versions expérimentales

Le feature engineering et la stratégie d’agrégation ont évolué avec les objectifs du projet. Dans les premières expériences, les données étaient représentées directement au niveau joueur : les informations issues des coups et des parties étaient agrégées pour construire un profil global du joueur, puis ce profil était utilisé pour prédire son Elo. Cette approche correspond à une logique d’*early aggregation*, où les parties sont résumées avant la prédiction.

Dans une deuxième étape, les données ont été séparées par cadence afin de mieux tenir compte des différences entre bullet, blitz et rapid. Cette séparation a conduit à raffiner les features utilisées, notamment avec des statistiques plus robustes comme la médiane, l’IQR et des taux lissés, afin de mieux fonctionner avec un nombre réduit de parties.

Enfin, dans les versions basées *game-level*, chaque partie est d’abord représentée séparément par ses propres features, puis le modèle prédit un Elo local pour cette partie.

Les prédictions obtenues sur plusieurs parties sont ensuite agrégées au niveau joueur. Cette approche correspond à une logique de *late aggregation* : au lieu de prédire le Elo à partir de features déjà agrégées, on prédit d’abord au niveau partie, puis on agrège les estimations. Cela permet d’analyser non seulement le niveau moyen estimé d’un joueur, mais aussi la dispersion de ses performances d’une partie à l’autre.

Le dataset final regroupe 1 886 profils joueur-cadence, chacun décrit par 219 variables couvrant la qualité des coups, la gestion du temps, la stabilité inter-parties et le style de jeu. Avant d’engager la phase de modélisation, il est nécessaire de mieux comprendre la structure de ces données : la distribution de la variable cible, les relations entre les features construites et le Elo, et les différences de comportement entre cadences. C’est l’objet du chapitre suivant.

Chapitre 3

Analyse exploratoire des données

Cette section présente une analyse exploratoire des données collectées selon trois niveaux de granularité : le joueur, la partie et le coup. L’objectif est d’étudier la distribution de la variable cible, de vérifier l’intérêt d’une séparation par cadence, et d’identifier les familles de variables les plus pertinentes avant la phase de modélisation.

3.1 Analyse de la variable cible : Elo

La variable cible du projet est le Elo du joueur. Dans le dataset final, elle est disponible pour 1 886 profils joueur-cadence. Les valeurs observées vont de 1004 à 2199, avec une moyenne d’environ 1653, une médiane de 1653.5 et un écart-type d’environ 302. La distribution est faiblement asymétrique, avec une skewness de -0.16, ce qui indique une distribution globalement équilibrée autour des niveaux intermédiaires.

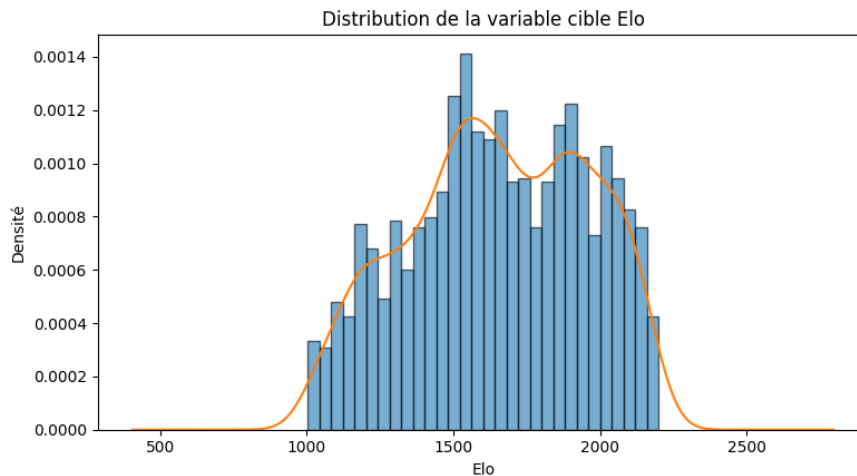


FIGURE 3.1 – Distribution globale du Elo dans le dataset final.

L’analyse par IQR ne détecte aucun outlier sur le Elo. Les bornes calculées sont environ 725 et 2613, ce qui englobe toutes les observations. Cela confirme que les valeurs de Elo collectées restent dans l’intervalle attendu du projet.

TABLE 3.1 – Statistiques descriptives de la variable cible Elo.

Statistique	Valeur
Nombre de profils	1 886
Elo minimum	1004
Elo médian	1653.5
Elo moyen	1652.7
Elo maximum	2199
Écart-type	302.5
Skewness	-0.16
Outliers IQR	0

Le Elo est une variable numérique continue. Le problème principal est donc formulé comme une régression supervisée. Les buckets Elo sont conservés pour les analyses stratifiées et les visualisations.

3.2 Répartition par cadence et par niveau

Les corpus bullet et blitz sont équilibrés : chaque cadence contient 700 joueurs, répartis en cinq buckets Elo de 140 joueurs chacun. Le corpus rapid est plus réduit avec 486 joueurs. Cette différence vient du filtrage plus strict : certains joueurs rapid ne possédaient pas suffisamment de parties classées, analysées et exploitables.

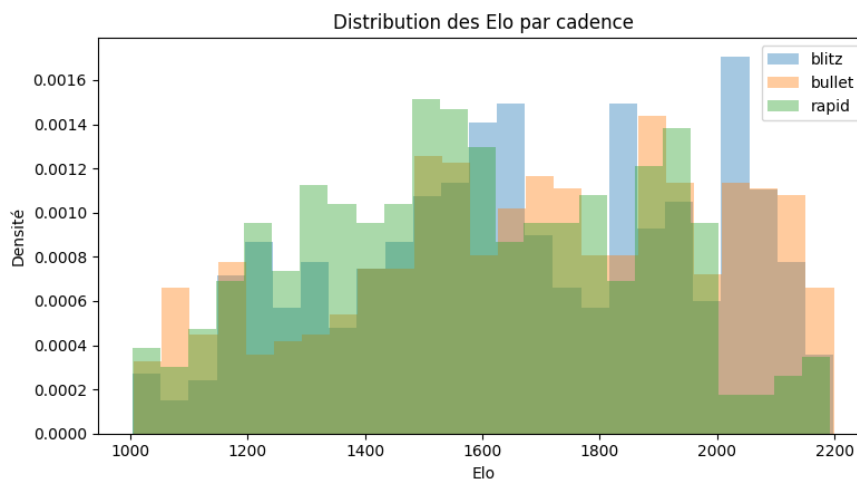


FIGURE 3.2 – Distribution du Elo par cadence.

La répartition par bucket confirme l'équilibre de bullet et blitz. Pour rapid, les buckets bas et intermédiaires restent représentés, mais le bucket 2000–2200 ne contient que 22 joueurs, ce qui doit être pris en compte lors de l'interprétation des résultats sur cette cadence.

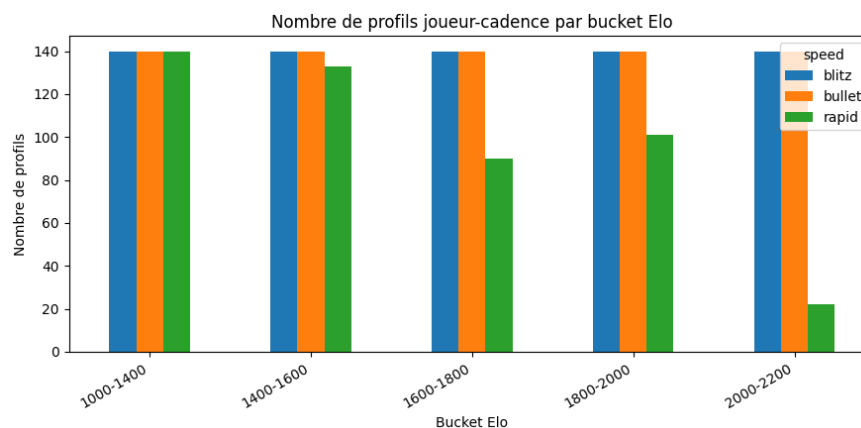


FIGURE 3.3 – Répartition des buckets Elo par cadence.

TABLE 3.2 – Répartition des données par cadence.

Cadence	Joueurs	Parties	Proportion des parties
Bullet	700	21 000	37.1 %
Blitz	700	21 000	37.1 %
Rapid	486	14 580	25.8 %

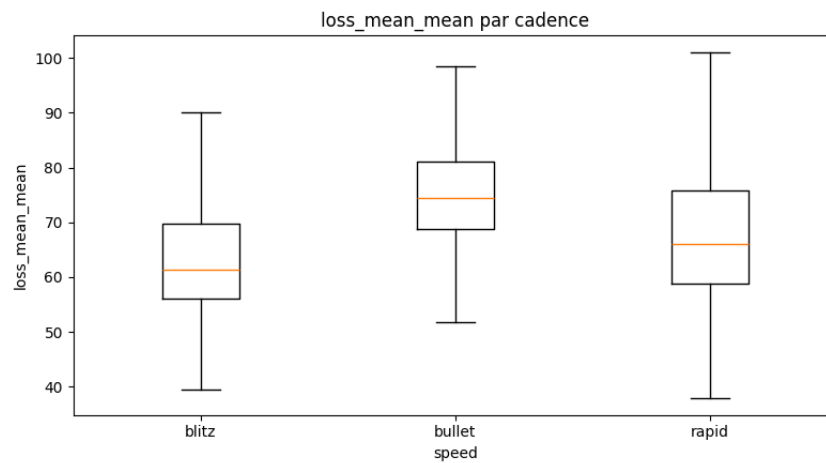
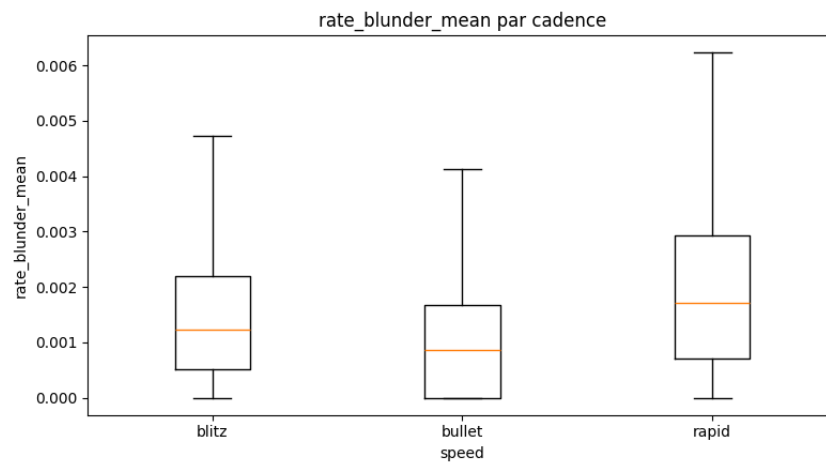
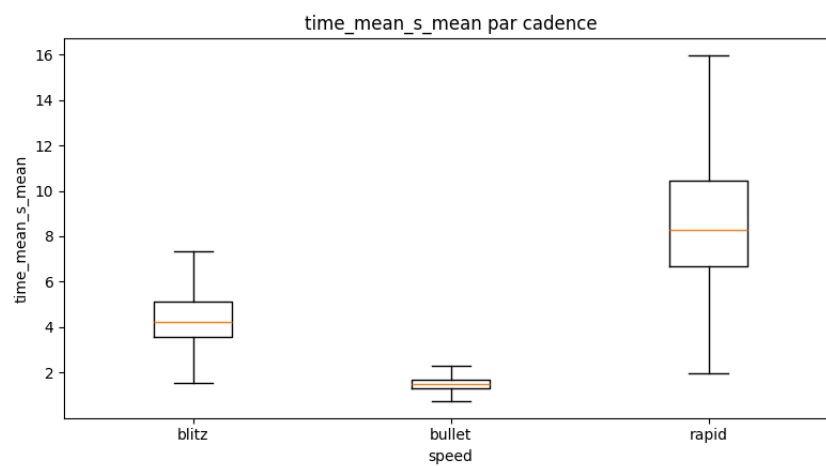
3.3 Pourquoi séparer les cadences ?

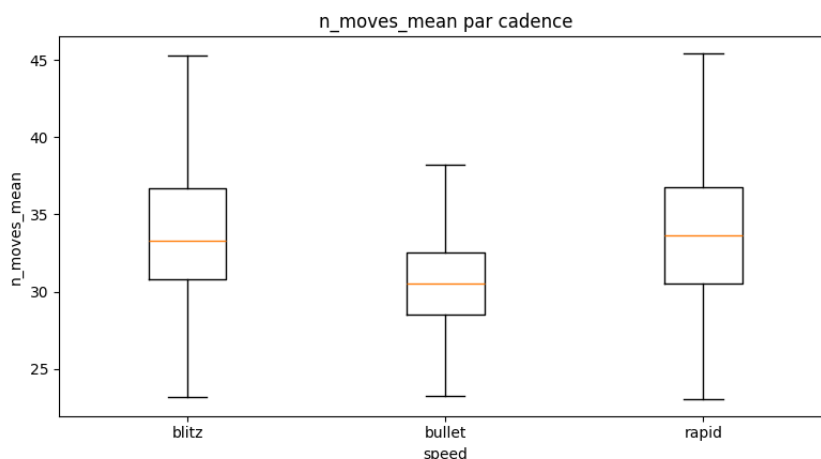
Les distributions des principales variables diffèrent fortement entre bullet, blitz et rapid. Les tests statistiques confirment ces différences : pour `loss_mean_mean`, `rate_blunder_mean`, `rate_error_mean`, `time_mean_s_mean` et `n_moves_mean`, les p-values des tests ANOVA et Kruskal-Wallis sont extrêmement faibles.

TABLE 3.3 – Tests statistiques des différences entre cadences.

Feature	p-value ANOVA	p-value Kruskal-Wallis
<code>loss_mean_mean</code>	7.43×10^{-89}	1.14×10^{-89}
<code>rate_blunder_mean</code>	2.61×10^{-30}	2.89×10^{-25}
<code>rate_error_mean</code>	4.84×10^{-45}	4.40×10^{-38}
<code>time_mean_s_mean</code>	0.0	0.0
<code>n_moves_mean</code>	1.16×10^{-64}	1.17×10^{-60}

Ces résultats montrent que les cadences ne diffèrent pas seulement par le temps disponible. Elles présentent aussi des différences significatives en qualité de coups, fréquence d'erreurs et longueur de parties. Une modélisation unique risquerait donc de mélanger des comportements propres à des contextes de jeu différents. Cette observation justifie l'analyse et l'entraînement de modèles séparés par cadence.

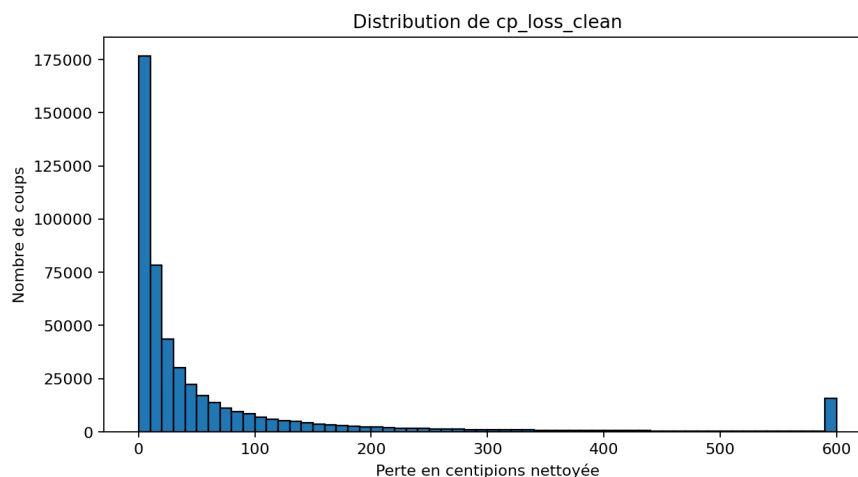
FIGURE 3.4 – Distribution de `loss_mean_mean` par cadence.FIGURE 3.5 – Distribution de `rate_blunder_mean` par cadence.FIGURE 3.6 – Distribution de `time_mean_s_mean` par cadence.

FIGURE 3.7 – Distribution de `n_moves_mean` par cadence.

3.4 Analyse de la qualité des coups

Distribution globale des erreurs

La qualité des coups est analysée à partir de `cp_loss_clean`, qui représente la perte en centipions associée au coup joué. La distribution est très déséquilibrée : la majorité des coups sont classés comme bons, tandis que les erreurs graves sont beaucoup plus rares.

FIGURE 3.8 – Distribution de `cp_loss_clean`.

Les grosses erreurs représentent une faible proportion des coups. Elles restent cependant très informatives : un seul blunder peut modifier fortement l'évaluation d'une position et influencer le résultat d'une partie.

Qualité par phase de jeu

Les pertes moyennes diffèrent selon les phases. L'ouverture présente les pertes les plus faibles, tandis que le milieu de jeu et la finale sont plus coûteux en centipions.

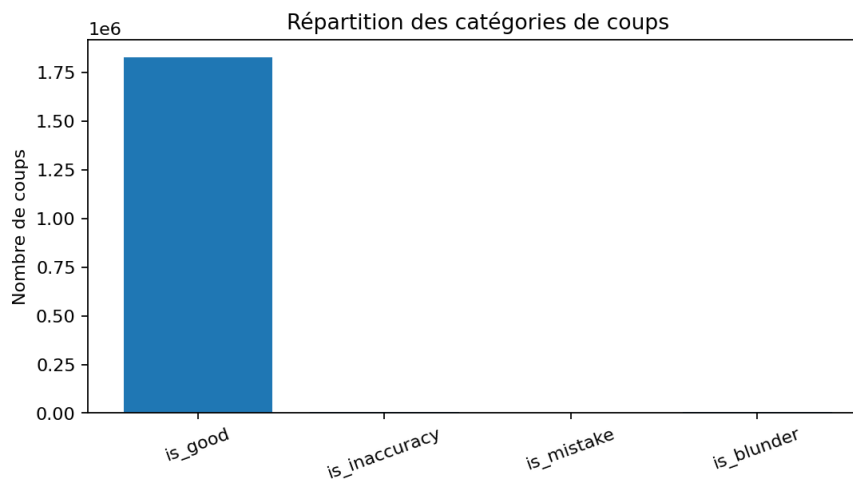


FIGURE 3.9 – Nombre de coups par catégorie de qualité.

TABLE 3.4 – Répartition des coups par catégorie de qualité.

Catégorie	Nombre de coups
Good	1 826 428
Inaccuracy	7 767
Mistake	2 148
Blunder	4 130

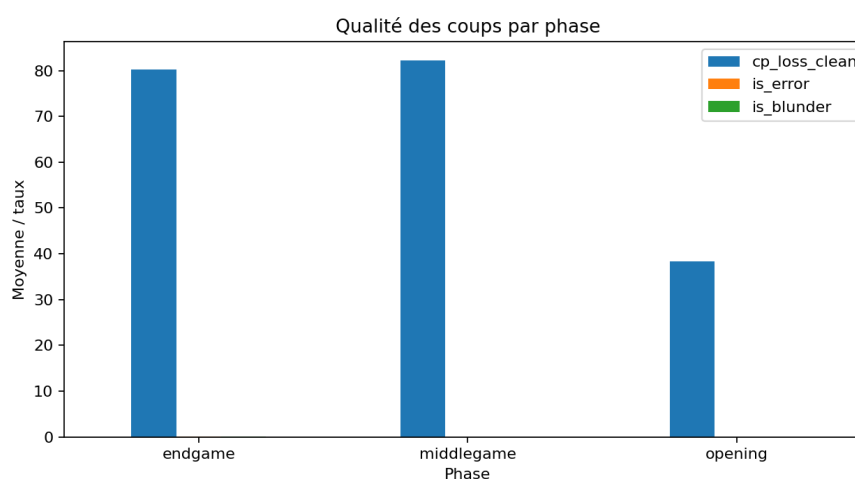


FIGURE 3.10 – Qualité moyenne des coups selon la phase de jeu.

TABLE 3.5 – Qualité des coups par phase de jeu.

Phase	cp_loss_clean moyen	Taux d'erreurs	Taux de blunders
Opening	38.35	0.11 %	0.005 %
Middlegame	82.19	0.68 %	0.15 %
Endgame	80.30	2.20 %	0.86 %

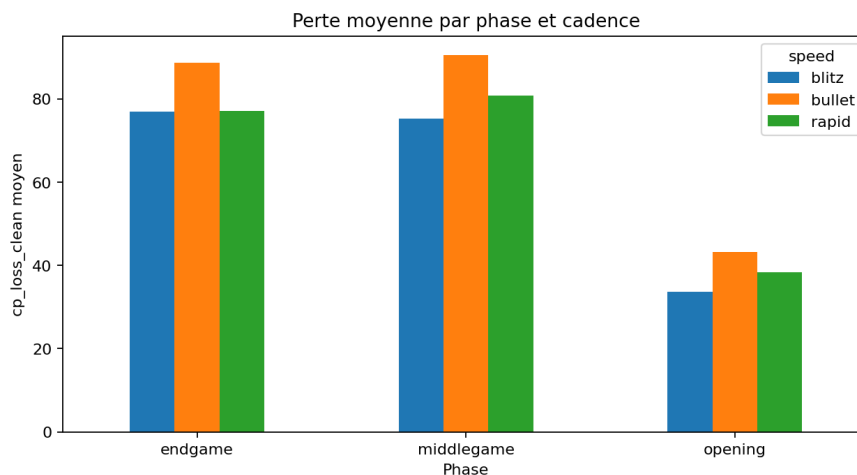


FIGURE 3.11 – Perte moyenne en centipions par phase et par cadence.

Par cadence, le bullet présente les pertes les plus élevées dans toutes les phases.

TABLE 3.6 – Perte moyenne en centipions par phase et par cadence.

Cadence	Opening	Middlegame	Endgame
Blitz	33.69	75.33	76.86
Bullet	43.13	90.60	88.74
Rapid	38.27	80.73	77.04

L'écart entre ouverture et phases suivantes montre que la performance d'un joueur ne peut pas être résumée uniquement par une perte moyenne globale. Les variables par phase, comme `loss_opening_mean`, `loss_middlegame_mean` et `loss_endgame_mean`, apportent donc une information complémentaire.

Qualité selon le contexte positionnel

Les variables de contexte montrent que les pertes en position équilibrée sont particulièrement informatives. Parmi les variables de contexte, `loss_when_equal_iqr`, `loss_when_equal_std` et `loss_when_equal_mean` sont les plus corrélées avec le Elo.

TABLE 3.7 – Corrélations des variables de contexte avec le Elo.

Feature	Corrélacion Spearman avec Elo
<code>loss_when_equal_iqr</code>	-0.371
<code>loss_when_equal_std</code>	-0.341
<code>loss_when_equal_mean</code>	-0.304
<code>loss_when_equal_median</code>	-0.222
<code>loss_when_losing_median</code>	-0.210

Cela suggère que la qualité des décisions dans les positions équilibrées est plus discriminante que dans les positions déjà très gagnantes ou très perdantes. Ce type de variable permet de mieux caractériser les contextes dans lesquels le joueur perd en précision.

Gestion du temps et pression temporelle

Une partie des coups possède un temps utilisé égal à zéro, ce qui peut correspondre à des coups joués instantanément, notamment des premoves ou des coups très rapides. Ce phénomène est plus fréquent en bullet.

TABLE 3.8 – Taux de coups avec temps utilisé nul par cadence.

Cadence	Taux de coups avec temps nul
Bullet	15.9 %
Blitz	12.2 %
Rapid	11.9 %

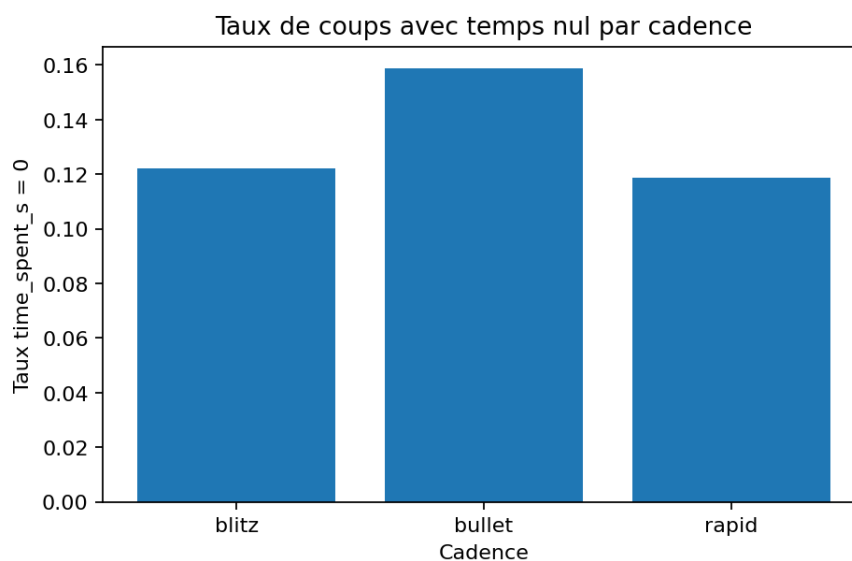


FIGURE 3.12 – Taux de coups avec temps nul par cadence.

L'analyse du temps restant avant le coup montre que la qualité diminue lorsque l'horloge devient faible. Dans les situations de temps très bas, le taux de blunders et la perte moyenne sont plus élevés. Par exemple, en blitz, le taux de blunders passe d'environ 0.036 % lorsque l'horloge est haute à environ 0.94 % lorsque le temps est très bas.

Ces observations justifient les variables qui croisent temps et qualité, comme `blunder_low_time_r`, `good_fast_move_rate` ou `time_pressure_by_phase`. Elles permettent de distinguer

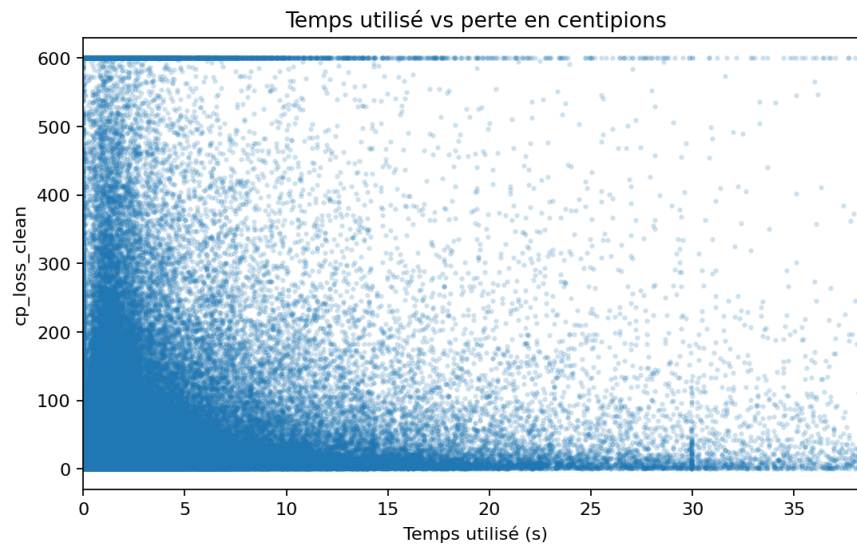


FIGURE 3.13 – Relation entre temps utilisé et perte en centipions.

les joueurs qui jouent vite efficacement de ceux qui deviennent instables sous pression temporelle.

3.5 Corrélations des features avec l'Elo

Les corrélations de Spearman montrent que les variables les plus liées au Elo sont principalement des variables de qualité de jeu et de stabilité. Les corrélations les plus fortes concernent notamment les pertes en ouverture, les pertes moyennes, les pertes élevées et la dispersion des pertes.

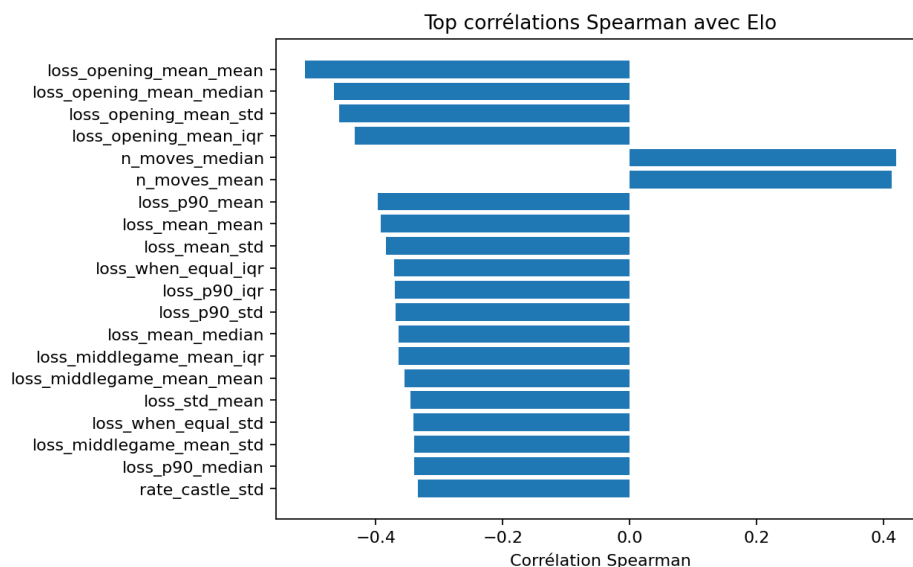


FIGURE 3.14 – Variables les plus corrélées avec le Elo selon Spearman.

TABLE 3.9 – Top corrélations de Spearman avec le Elo.

Feature	Corrélation Spearman avec Elo
loss_opening_mean_mean	-0.511
loss_opening_mean_median	-0.466
loss_opening_mean_std	-0.457
loss_opening_mean_iqr	-0.433
n_moves_median	0.420
n_moves_mean	0.412
loss_p90_mean	-0.397
loss_mean_mean	-0.392
loss_mean_std	-0.384
loss_when_equal_iqr	-0.371

Les corrélations négatives signifient que les pertes diminuent lorsque le Elo augmente. Les joueurs plus forts ont donc tendance à perdre moins de centipions par coup et à être plus réguliers. Les variables de dispersion, comme les écarts-types et IQR, apparaissent aussi parmi les variables corrélées, ce qui montre que la stabilité inter-parties est informative.

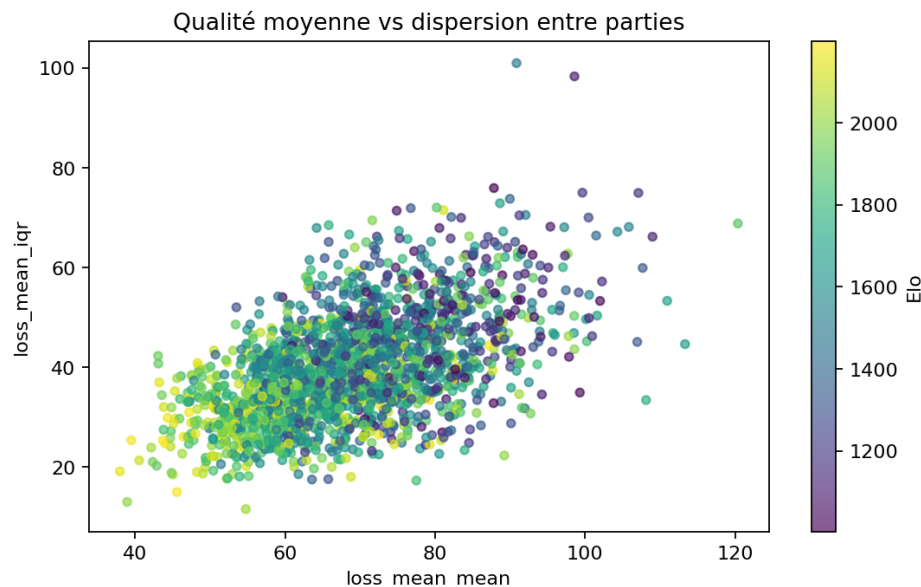


FIGURE 3.15 – Relation entre perte moyenne et dispersion des pertes.

Les tests par bucket Elo confirment que certaines variables discriminent bien les niveaux. Par exemple, `loss_mean_mean`, `win_rate` et `time_mean_s_mean` présentent des différences significatives entre buckets. En revanche, dans cette analyse globale, `rate_blunder_mean` et `rate_error_mean` ne ressortent pas significativement, probablement parce que les blunders sont rares et parce que leur effet dépend fortement du contexte.

Ces corrélations restent toutefois une première indication. Certaines relations peuvent être non linéaires ou dépendre d'interactions entre plusieurs variables, ce qui motive l'utilisation de modèles plus flexibles dans la phase de modélisation.

3.6 Multicolinéarité, redondance et fuite d'information

La matrice de corrélation met en évidence plusieurs groupes de variables très redondantes. Certaines paires ont une corrélation presque parfaite.

TABLE 3.10 – Exemples de paires de variables fortement corrélées.

Variable 1	Variable 2	Corrélation absolue
<code>rate_good_mean</code>	<code>rate_error_mean</code>	1.000
<code>rate_good_median</code>	<code>rate_error_median</code>	1.000
<code>rate_blunder_iqr</code>	<code>n_blunders_iqr</code>	0.999
<code>time_mean_s_mean</code>	<code>time_mean_s_median</code>	0.993
<code>loss_mean_mean</code>	<code>loss_p90_mean</code>	0.965

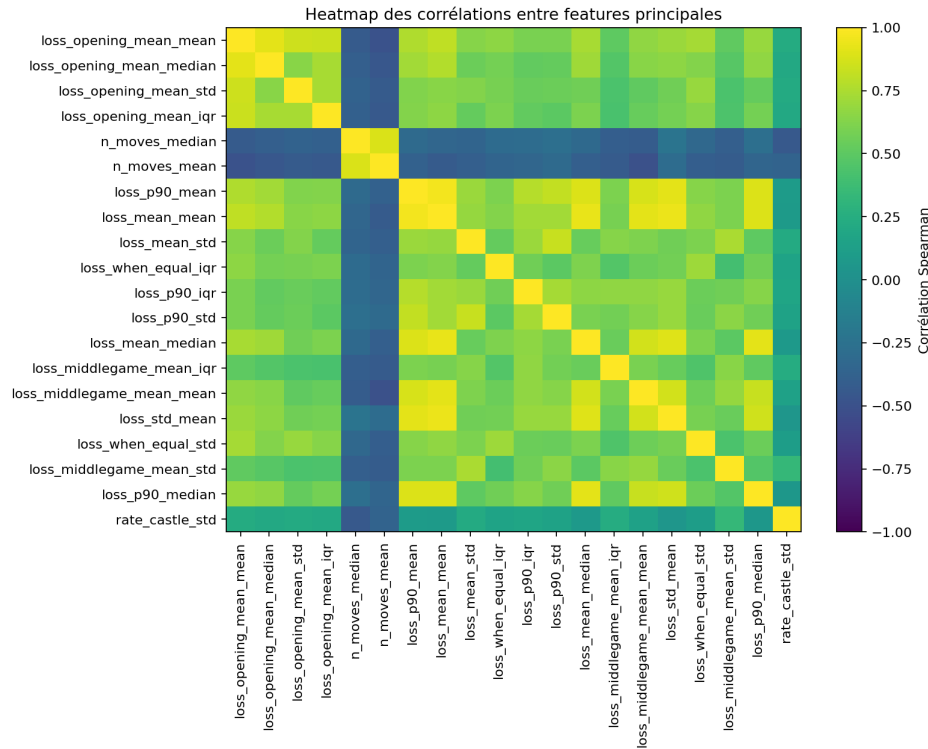


FIGURE 3.16 – Matrice de corrélation des principales features.

Le calcul du VIF confirme aussi une forte multicolinéarité sur certaines variables de perte. Par exemple, `loss_mean_mean`, `loss_p90_mean` et `loss_opening_mean_mean` ont des VIF élevés.

Cette redondance n'est pas forcément problématique pour des modèles de boosting comme LightGBM, mais elle doit être prise en compte pour les modèles linéaires comme Ridge. Elle justifie aussi une sélection ou une régularisation des features dans les expériences.

Enfin, `elo_bucket` est dérivé directement de `elo`. Il est donc exclu des variables explicatives afin d'éviter une fuite d'information.

3.7 Stabilité des features selon le nombre de parties

L'objectif du projet est d'estimer rapidement le niveau d'un joueur. Il est donc nécessaire d'étudier la stabilité des features lorsque le nombre de parties disponibles diminue. L'analyse few-shot compare les statistiques calculées avec 3, 5, 10, 20 et 30 parties à la valeur de référence calculée avec toutes les parties disponibles.

Les erreurs d'estimation diminuent lorsque le nombre de parties augmente. Certaines variables sont plus stables que d'autres. Par exemple, avec seulement 3 parties, `loss_median` est plus stable que `loss_mean`, tandis que `loss_p90` reste plus variable.

Cette analyse justifie l'utilisation de statistiques robustes comme la médiane, l'IQR

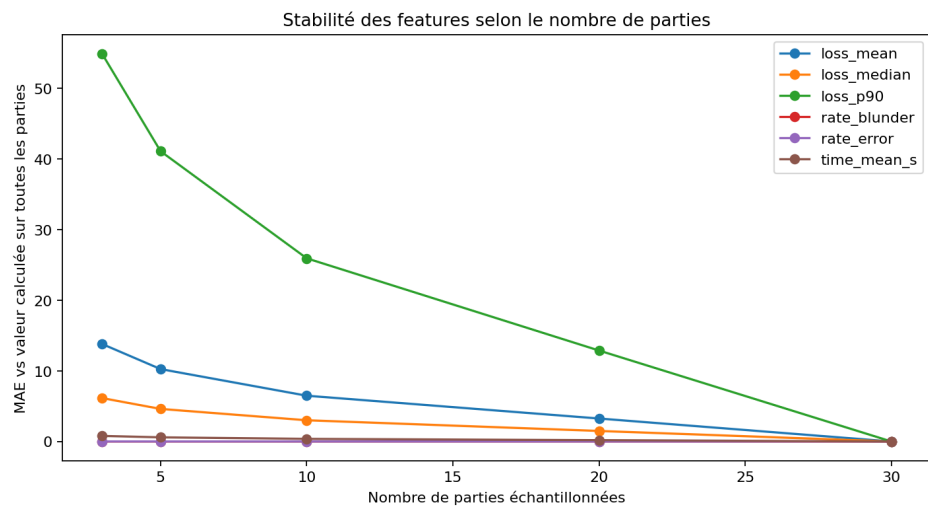


FIGURE 3.17 – Stabilité des features selon le nombre de parties disponibles.

TABLE 3.11 – Erreur d’estimation des statistiques few-shot par rapport à la référence.

Nombre de parties	MAE de la moyenne vs référence	MAE de la médiane vs référence
3	13.84	15.64
5	10.24	11.92
10	6.48	7.56
20	3.26	3.78
30	0.00	0.00

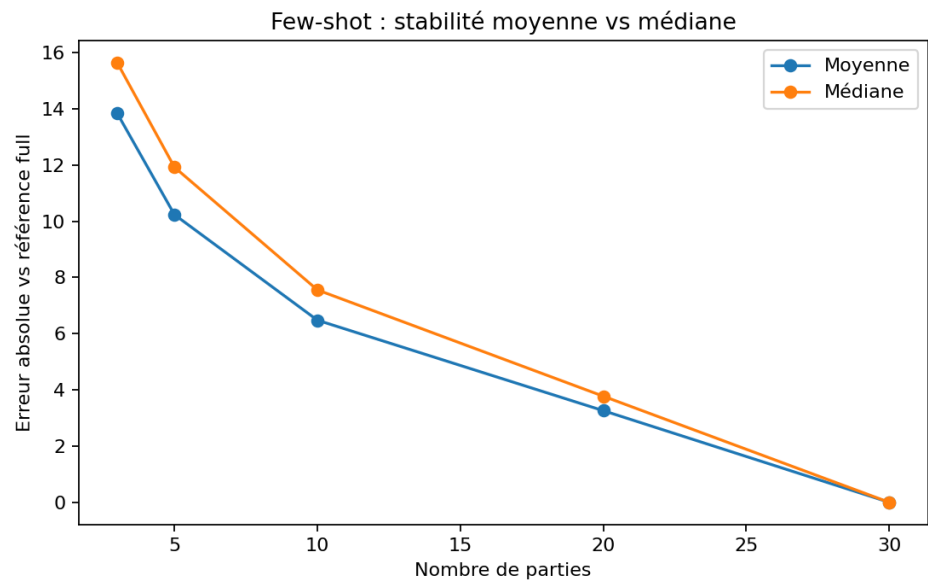


FIGURE 3.18 – Comparaison de la stabilité de la moyenne et de la médiane en contexte few-shot.

ou les taux lissés dans les versions récentes du feature engineering. Avec peu de parties, les features peuvent être sensibles au choix des parties observées ; il est donc nécessaire de construire des variables moins instables.

L'ensemble de ces analyses confirme la pertinence des choix retenus pour la suite du projet. La variable cible est bien distribuée et ne présente pas d'outliers problématiques. Les cadences se comportent de manière statistiquement distincte, ce qui justifie l'entraînement de modèles séparés. La qualité des coups, leur stabilité inter-parties et le comportement sous pression temporelle apparaissent comme les dimensions les plus discriminantes du niveau Elo. Enfin, l'analyse few-shot montre que certaines features restent exploitables avec un nombre réduit de parties, ce qui est cohérent avec l'objectif de prédiction rapide du niveau. Ces observations orientent directement les choix de modélisation : le choix des algorithmes, la stratégie d'évaluation par cadence, et la sélection des features les plus stables et les plus informatives sont abordés dans la section suivante.

Chapitre 4

Modélisation prédictive d'elo

Ce chapitre présente les expériences de prédiction du Elo menées dans le projet. L'objectif est d'estimer le niveau d'un joueur à partir de ses parties analysées, en comparant plusieurs modèles et plusieurs stratégies de construction des features. Les expériences sont présentées de manière progressive : on part d'une baseline globale avec beaucoup de parties, puis on raffine les données par cadence et par sélection de variables.

4.1 Approche générale

L'objectif de la modélisation est de prédire le niveau Elo d'un joueur à partir des informations extraites de ses parties. Le problème est traité comme une tâche de régression supervisée : les variables d'entrée décrivent le comportement du joueur, tandis que la variable cible correspond à son Elo dans une cadence donnée.

Pour chaque joueur et chaque cadence (*bullet*, *blitz* ou *rapid*), on dispose d'un ensemble de N parties. Chaque partie est décrite par un vecteur de variables calculées à partir des coups, du temps, des erreurs et du contexte de jeu. Ces informations peuvent ensuite être résumées au niveau joueur par une fonction d'agrégation :

$$x_{j,c} = \phi(\{g_1, g_2, \dots, g_N\}) \in \mathbb{R}^p \quad (4.1)$$

où g_i représente les variables de la i -ème partie, et ϕ désigne une fonction d'agrégation comme la moyenne, la médiane, l'écart-type ou l'IQR. Le modèle apprend alors une fonction de prédiction :

$$\hat{y}_{j,c} = \hat{f}(x_{j,c}) \approx y_{j,c} \quad (4.2)$$

où $y_{j,c}$ est le Elo réel du joueur j dans la cadence c .

Dans l'expérience basée sur la prédiction au niveau partie, la logique est légèrement différente. Le modèle prédit d'abord un Elo estimé pour chaque partie, puis les

prédictions obtenues sur les N parties du joueur sont agrégées :

$$\hat{y}_{j,c}^{(i)} = \hat{f}_{game}(g_i) \quad (4.3)$$

$$\hat{y}_{j,c} = \text{Agg} \left(\hat{y}_{j,c}^{(1)}, \dots, \hat{y}_{j,c}^{(N)} \right) \quad (4.4)$$

Cette approche permet de conserver une information sur la variabilité des performances d'une partie à l'autre, au lieu de résumer directement toutes les parties avant la prédiction.

4.1.1 Protocole d'évaluation

Le dataset est séparé en trois ensembles : entraînement, validation et test. La séparation est réalisée au niveau joueur, avec une stratification par tranche Elo afin de conserver une distribution équilibrée des niveaux dans chaque ensemble. La répartition utilisée est de 64 % pour l'entraînement, 16 % pour la validation et 20 % pour le test.

TABLE 4.1 – Répartition des joueurs par cadence et ensemble.

Cadence	Total	Train	Val.	Test	Ratio
Bullet	700	448	112	140	64/16/20
Blitz	700	448	112	140	64/16/20
Rapid	486	310	78	98	64/16/20

Afin d'évaluer la robustesse des modèles avec peu de données, plusieurs valeurs de N sont testées, notamment $N \in \{3, 5, 10, 30\}$. Pour chaque valeur de N , plusieurs tirages aléatoires de parties sont réalisés. Cela permet de mesurer à la fois la performance moyenne et la stabilité du modèle selon les parties disponibles.

4.1.2 Métriques d'évaluation

Trois métriques sont utilisées pour comparer les modèles.

La métrique principale est la MAE (*Mean Absolute Error*), qui mesure l'erreur moyenne en points Elo :

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j| \quad (4.5)$$

Elle est directement interprétable : une MAE de 150 signifie que le modèle se trompe en moyenne de 150 points Elo.

Le RMSE (*Root Mean Squared Error*) est utilisé comme métrique secondaire :

$$RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2} \quad (4.6)$$

Il pénalise davantage les grandes erreurs et permet donc d'identifier les modèles qui produisent parfois des prédictions très éloignées du vrai Elo.

Enfin, le coefficient de détermination R^2 mesure la part de variance du Elo expliquée par le modèle :

$$R^2 = 1 - \frac{\sum_{j=1}^n (y_j - \hat{y}_j)^2}{\sum_{j=1}^n (y_j - \bar{y})^2} \quad (4.7)$$

Un R^2 proche de 1 indique que le modèle explique bien les différences de niveau entre joueurs, tandis qu'un R^2 proche de 0 signifie qu'il ne fait pas beaucoup mieux qu'une prédiction moyenne.

4.1.3 Modèles testés

Six familles de modèles sont comparées, allant des modèles linéaires aux architectures neuronales profondes. Elles sont présentées ici par ordre croissant de complexité.

Ridge – Régression linéaire régularisée. La régression Ridge minimise la fonction objectif suivante :

$$\hat{\beta} = \arg \min_{\beta} (\|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2) \quad (4.8)$$

Le terme de régularisation $\lambda \|\beta\|_2^2$ pénalise les coefficients trop grands et réduit le surapprentissage. Ridge est utilisé comme baseline linéaire : si les modèles plus complexes ne font pas mieux, cela signifie que les relations entre les features et le Elo sont essentiellement linéaires.

Random Forest – Forêt d'arbres de décision. Un Random Forest construit T arbres de décision entraînés sur des sous-échantillons aléatoires des données, selon le principe du *bagging*, ainsi que sur des sous-ensembles aléatoires de features à chaque nœud. La prédiction finale est obtenue en moyennant les prédictions des T arbres. Ce mécanisme de double randomisation réduit la variance et améliore la robustesse face aux features bruitées.

XGBoost – Gradient boosting avec régularisation. XGBoost construit des arbres de manière séquentielle : chaque nouvel arbre corrige les erreurs résiduelles du précédent. Il intègre une régularisation L1 et L2 sur les poids des feuilles, ce qui le rend moins sensible au surapprentissage que le boosting classique. La fonction objectif

optimisée peut s'écrire :

$$\mathcal{L} = \sum_j \ell(y_j, \hat{y}_j) + \sum_t \Omega(f_t) \quad (4.9)$$

où ℓ désigne la fonction de perte, généralement quadratique dans notre cas, et où :

$$\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (4.10)$$

pénalise la complexité de chaque arbre f_t .

LightGBM – Gradient boosting optimisé. LightGBM est une implémentation optimisée du gradient boosting. Contrairement à une croissance classique des arbres niveau par niveau, LightGBM utilise une croissance *leaf-wise*, c'est-à-dire qu'il développe en priorité la feuille qui réduit le plus la perte. Cette stratégie produit des arbres plus asymétriques, mais souvent plus précis pour un même nombre de feuilles, avec un coût mémoire et temporel réduit. Dans ce travail, LightGBM constitue l'un des modèles principaux, avec des hyperparamètres optimisés par Optuna.

CatBoost – Gradient boosting pour variables catégorielles. CatBoost est une autre variante du gradient boosting, conçue notamment pour mieux gérer les variables catégorielles grâce à un encodage cible ordonné. Dans notre cas, les features sont principalement numériques, mais CatBoost est tout de même évalué comme modèle de comparaison, car il est robuste sur les données tabulaires et limite naturellement certains effets de surapprentissage.

DeepSets et Attention – Architectures neuronales sur ensembles. Ces architectures sont utilisées pour exploiter directement un ensemble de parties d'un même joueur. Chaque partie g_i est encodée en un vecteur latent :

$$h_i = \text{Enc}(g_i) \in \mathbb{R}^d \quad (4.11)$$

où Enc désigne un réseau de neurones chargé de transformer les features d'une partie en représentation compacte. Les représentations des N parties sont ensuite agrégées pour obtenir un embedding joueur :

$$z_{j,c} = \text{Agg}(h_1, h_2, \dots, h_N) \quad (4.12)$$

Deux modes d'agrégation sont comparés :

— **DeepSets Mean :**

$$z_{j,c} = \frac{1}{N} \sum_{i=1}^N h_i \quad (4.13)$$

c'est-à-dire une moyenne simple des embeddings des parties.

— **DeepSets Attention** :

$$z_{j,c} = \sum_{i=1}^N \alpha_i h_i \quad (4.14)$$

où les poids α_i sont appris par un mécanisme d'attention, permettant de donner plus d'importance aux parties les plus informatives.

L'embedding final $z_{j,c}$ est ensuite concaténé aux features manuelles V8, puis fourni en entrée à un modèle LightGBM. Cette combinaison permet d'exploiter à la fois des représentations apprises automatiquement et des variables comportementales interprétables.

4.2 Résultats et discussion

Dans cette section, nous présentons les différentes expériences de modélisation réalisées, ainsi que les résultats obtenus. Les expériences suivent une progression logique : d'abord une baseline générale, puis une séparation par cadence, ensuite l'ajout d'embeddings neuronaux, et enfin une approche basée sur la prédiction au niveau partie.

4.3 Expérience 1 – Baseline globale et effet du nombre de parties

4.3.1 Description

Cette première expérience sert de point de référence. Elle utilise un corpus global d'environ 1 000 joueurs avec jusqu'à 100 parties par joueur, sans séparation stricte par cadence. L'objectif est de vérifier si les features comportementales permettent déjà de prédire correctement le Elo lorsque l'on dispose d'un volume important de parties.

Les modèles testés sont des modèles tabulaires classiques : Ridge, CatBoost, XGBoost, LightGBM et une combinaison par stacking. Le stacking est seulement utilisé comme comparaison haute, car il combine plusieurs modèles au lieu de s'appuyer sur un seul prédicteur.

4.3.2 Résultats

TABLE 4.2 – Résultats principaux de l’expérience 1 sur le corpus 1 000 joueurs $\times$ 100 parties.

Modèle	MAE	RMSE	R^2	Remarque
CatBoost	109.2	143.3	0.802	Meilleur modèle individuel.
XGBoost tuned	112.7	142.7	0.803	Très proche de CatBoost.
Ridge	117.4	146.8	0.792	Baseline linéaire solide.
OOF Blend	102.7	131.8	0.832	Moyenne des prédictions issues des modèles.
Stacking Ridge / ElasticNet	100.1	128.8–128.9	0.840	Meilleure performance globale.

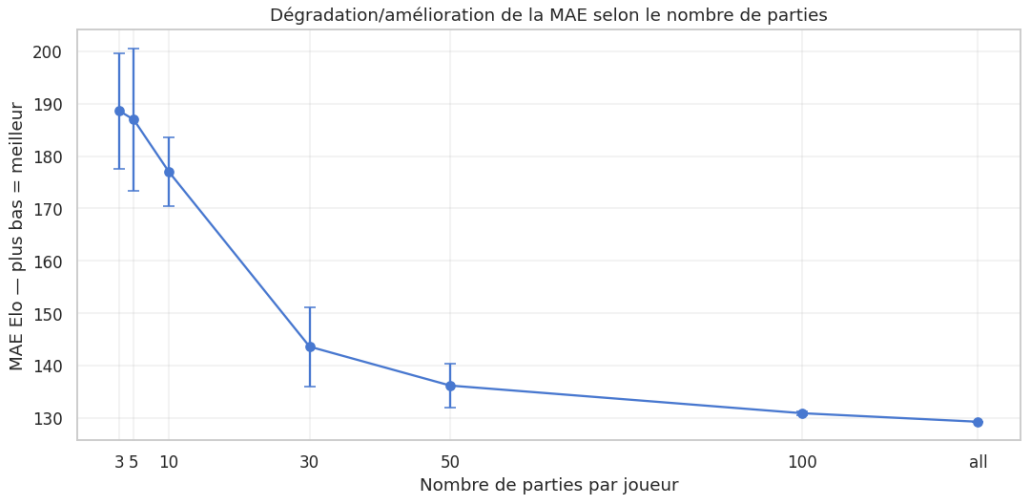


FIGURE 4.1 – Évolution de la MAE selon le nombre de parties disponibles par joueur.

4.3.3 Discussion

Les résultats montrent que la prédiction du Elo est possible lorsque le nombre de parties est suffisant. Le meilleur score est obtenu par le stacking, avec une MAE proche de 100 points Elo, ce qui constitue une borne haute pour cette première phase. Les modèles de gradient boosting restent les plus efficaces parmi les modèles individuels, ce qui confirme leur intérêt pour des features tabulaires hétérogènes. La figure 4.1 montre aussi que la performance se dégrade quand le nombre de parties diminue : l’estimation devient donc plus difficile en contexte few-shot.

4.4 Expérience 2 – Séparation par cadence et sélection de features

4.4.1 Description

Comme indiqué dans les objectifs du projet et dans l'analyse exploratoire, les comportements ne sont pas identiques selon la cadence : les features liées au temps dominent davantage en bullet et blitz, tandis que la qualité des coups devient plus importante en rapid. Pour cette raison, les modèles sont entraînés séparément pour chaque cadence.

Cette expérience utilise une version raffinée des features V8. Plusieurs nombres de parties ont été testés ($N = 3, 5, 10, 30$), mais la présentation se concentre sur $N = 10$ parties : c'est le meilleur compromis entre peu de données disponibles et métriques encore acceptables. Avec 3 ou 5 parties, l'estimation est trop instable ; avec 30 parties, les résultats sont meilleurs mais le cas d'usage est moins réaliste pour une estimation rapide.

Quatre stratégies de sélection sont comparées : garder toutes les features, retirer les features peu utiles ou négatives, supprimer les blocs Contexte/Stabilité, ou ne garder qu'un noyau de features robustes.

4.4.2 Résultats

TABLE 4.3 – Comparaison des stratégies de sélection de features, $N = 10$ parties.

Exp.	Principe	Cadence	Features	Meilleur modèle	MAE	MAE std	R^2	R^2 std
A	Toutes les features V8	bullet	215/215	LightGBM	175.8	12.0	0.522	0.047
A	Toutes les features V8	blitz	215/215	LightGBM	159.5	7.1	0.547	0.049
A	Toutes les features V8	rapid	215/215	XGBoost	152.1	11.2	0.532	0.069
B	Suppression des features négatives	bullet	117/215	Ridge	174.9	13.6	0.509	0.065
B	Suppression des features négatives	blitz	133/215	XGBoost/LightGBM	156.2	5.9	0.567	0.043
B	Suppression des features négatives	rapid	92/215	XGBoost / LightGBM	153.3	11.9 / 11.2	0.521 / 0.525	0.073 / 0.069
C	Retrait Contexte + Stabilité	bullet	189/215	LightGBM	178.1	13.3	0.501	0.058
C	Retrait Contexte + Stabilité	blitz	189/215	LightGBM	164.4	7.7	0.520	0.054
C	Retrait Contexte + Stabilité	rapid	189/215	CatBoost	159.6	10.4	0.488	0.063
D	Features robustes seulement	bullet	45/215	Ridge	189.2	29.6	-0.432	3.414
D	Features robustes seulement	blitz	45/215	XGBoost	167.0	9.6	0.503	0.068
D	Features robustes seulement	rapid	45/215	Ridge	150.5	10.4	0.534	0.059

4.4.3 Discussion

Globalement, la stratégie B est la plus intéressante : elle réduit le nombre de features et obtient les meilleurs résultats en blitz, tout en restant compétitive en bullet et en rapid. Cela suggère que certaines variables ajoutent du bruit ou des signaux faibles qui n'aident pas la généralisation. La stratégie D est trop restrictive : elle peut bien fonctionner en rapid, mais elle dégrade fortement bullet et blitz. Pour la suite, l'idée retenue est donc de conserver une représentation riche, mais en contrôlant les features peu utiles selon la cadence.

4.5 Expérience 3 – Ajout d’embeddings neuronaux

4.5.1 Description

Cette expérience teste si une représentation neuronale des parties peut compléter les features sélectionnées dans l’expérience précédente. Dans notre cas, un embedding est un petit vecteur numérique appris automatiquement par un réseau : il résume une partie, ou un ensemble de parties, en capturant des régularités qui ne sont pas toujours visibles dans les features manuelles. Deux variantes sont testées : DeepSets avec moyenne simple, et DeepSets avec attention. Le principe retenu est de ne pas remplacer les features de l’expérience 2, mais de les enrichir. Les embeddings obtenus à partir des parties sont concaténés aux features classiques, puis l’ensemble est donné à un modèle LightGBM.

4.5.2 Résultats

Les résultats ci-dessous correspondent à l’expérience 3 appliquée à la baseline de l’expérience 2 (B) pruned. Le tableau présente les meilleurs résultats obtenus pour chaque cadence et pour différents nombres de parties. Le cas $N = 10$ reste le point de comparaison principal avec les autres expériences, car il représente un compromis réaliste entre peu de parties disponibles et performance acceptable.

TABLE 4.4 – Meilleures configurations l’expérience 3 avec embeddings, pour $N = 10$ parties.

Cadence	N parties	Meilleure approche	MAE	R^2	Runs
Blitz	10	V8-B + embedding mean + LightGBM	159.7 ± 4.7	0.583	3
Bullet	10	V8-B + embedding mean + LightGBM	174.3 ± 7.6	0.551	3
Rapid	10	V8-B + embedding attention + LightGBM	137.4 ± 11.0	0.578	3

Une version complémentaire de cette expérience, exécutée avec plusieurs seeds sur la baseline, confirme la même tendance générale.

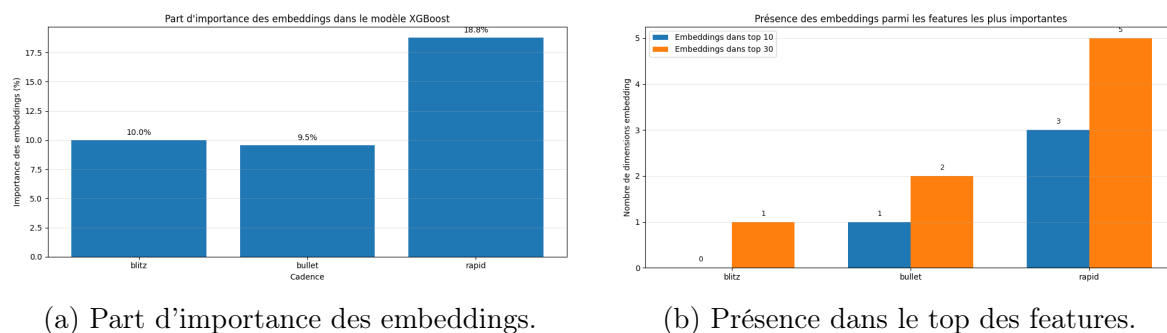


FIGURE 4.2 – Importance des embeddings dans le modèle LightGBM selon la cadence.

4.5.3 Discussion

Les embeddings apportent un signal complémentaire aux autres features, mais ils ne remplacent pas les variables manuelles. Leur intérêt est surtout visible en rapid, où ils représentent une part plus importante de l'information utilisée par LightGBM et apparaissent davantage parmi les features les plus importantes. En bullet et blitz, l'apport reste plus limité, probablement parce que les variables de temps et de qualité capturent déjà une grande partie du comportement utile. Cette expérience valide donc l'idée d'enrichir le profil joueur par des représentations apprises, tout en gardant une approche tabulaire interprétable basée sur LightGBM.

4.6 Expérience 4 – Late aggregation et prédiction au niveau partie

4.6.1 Description

Contrairement aux expériences précédentes, qui prédisaient le Elo à partir d'un profil joueur déjà agrégé, cette expérience descend d'abord au niveau partie. Chaque partie reçoit une estimation locale du Elo, puis les prédictions obtenues sur les N parties sont agrégées pour produire le Elo final du joueur. Cette logique correspond à une approche de *late aggregation*.

Dans cette expérience, le prédicteur partie est un LightGBM entraîné sur des features *game-level*. Les prédictions partie-niveau sont ensuite agrégées par moyenne, médiane, moyenne tronquée ou moyenne pondérée par le logarithme du nombre de coups. Comme les prédictions brutes présentaient une régression vers la moyenne, une calibration Ridge a été ajoutée à partir des statistiques des prédictions agrégées, notamment la moyenne, la dispersion et les quartiles.

On a repris la même idée est l'enrichie avec des embeddings appris à partir des coups : les coups sont encodés en représentation de partie, puis combinés aux features *game-level*. La comparaison ci-dessous se concentre sur $N = 10$ parties, qui reste le compromis principal retenu dans le rapport.

4.6.2 Résultats

TABLE 4.5 – Résultats sans embeddings à $N = 10$ parties.

Cadence	Agrégateur	Calibration	MAE brute	MAE calibrée	R^2 calibré
Blitz	Weighted log n_{moves}	Ridge	188.1 ± 2.7	165.0 ± 6.9	0.537
Bullet	Mean	Ridge	183.2 ± 4.1	154.7 ± 8.4	0.574
Rapid	Weighted log n_{moves}	Ridge	182.7 ± 2.4	148.9 ± 4.2	0.566

TABLE 4.6 – Résultats avec embeddings à $N = 10$ parties.

Cadence	Meilleure approche	MAE	R^2	Biais moyen
Blitz	calibrated weighted log n_{moves}	150.6 ± 6.7	0.592	7.13
Bullet	calibrated median	181.3 ± 16.8	0.460	-1.60
Rapid	calibrated weighted log n_{moves}	161.5 ± 3.2	0.434	17.31

4.6.3 Discussion

La calibration améliore nettement V10 : à $N = 10$, elle réduit la MAE de 188.1 à 165.0 en blitz, de 183.2 à 154.7 en bullet et de 182.7 à 148.9 en rapid. la version sans embeddings devient donc compétitive avec les meilleures approches précédentes, surtout en bullet et en rapid. La version avec embeddings apporte un gain clair en blitz, où l'embedding de coups descend à 150.6 de MAE, mais il ne généralise pas mieux en bullet ni en rapid. L'intérêt principal de cette expérience n'est donc pas seulement le score brut : elle permet aussi d'obtenir une estimation partie par partie et d'analyser la régularité du joueur à partir de la dispersion des Elo prédits.

4.7 Synthèse des expériences de modélisation

Les expériences menées montrent une progression claire dans la manière de prédire le Elo. L'expérience 1 établit une première baseline solide : avec un volume important de parties par joueur, les modèles de gradient boosting et le stacking atteignent les meilleurs résultats globaux. Cependant, cette configuration reste moins réaliste, car elle suppose un grand nombre de parties disponibles pour chaque joueur.

L'expérience 2 est plus proche de notre objectif pratique. En séparant les données par cadence et en raffinant les features comportementales, on observe que les variables prédictives ne sont pas les mêmes en bullet, blitz et rapid. La configuration basée sur la suppression des features peu utiles ou négatives donne globalement le meilleur compromis : elle réduit le bruit, améliore surtout les résultats en blitz, et reste compétitive sur les autres cadences.

L'expérience 3 reprend les features raffinées de l'expérience 2 et les enrichit avec des embeddings neuronaux. Dans notre cas, un embedding correspond à une représentation numérique apprise automatiquement à partir des parties, afin de capturer des informations qui ne sont pas forcément visibles dans les features manuelles. Cette approche apporte une information complémentaire, surtout en rapid, où les embeddings sont davantage utilisés par le modèle. À $N = 10$ parties, elle donne les meilleurs résultats pour le rapid, tandis que le gain reste plus modéré en bullet et blitz.

L'expérience 4 explore une autre stratégie : au lieu de prédire directement le Elo à partir d'un profil joueur agrégé, le modèle prédit d'abord un Elo au niveau de chaque partie, puis agrège ces prédictions pour obtenir le niveau final du joueur. Cette approche est intéressante car elle exploite davantage d'exemples d'entraînement et permet aussi d'observer la régularité du joueur d'une partie à l'autre. Même si elle ne surpasse pas toujours l'expérience 3 en MAE brute, elle ouvre une piste importante pour analyser les variations de performance et construire des profils plus fins.

Au final, le meilleur choix dépend de la cadence et du nombre de parties disponibles. Avec peu de parties, les features comportementales raffinées restent essentielles, notamment les variables liées au temps et à la qualité des coups. Pour les cadences plus longues, l'ajout d'embeddings devient plus intéressant, car les parties contiennent davantage d'informations exploitables. Ces résultats confirment que le Elo peut être estimé avec un nombre limité de parties, mais aussi que le niveau seul ne suffit pas à décrire complètement un joueur.

Cette partie a montré qu'il est possible d'estimer le niveau Elo d'un joueur à partir d'un petit nombre de parties, en combinant des features comportementales, une séparation par cadence et, dans certains cas, des représentations apprises automatiquement. Cette prédiction du niveau constitue une première étape : elle permet de situer le joueur sur une échelle de force. Cependant, deux joueurs ayant un Elo proche peuvent avoir des manières de jouer très différentes. La suite du travail consiste donc à dépasser la simple estimation du niveau pour étudier les profils comportementaux : à Elo comparable, peut-on distinguer plusieurs styles de joueurs selon leur gestion du temps, leur stabilité, leur agressivité ou leurs erreurs typiques ?

Chapitre 5

Profil comportemental des joueurs

Cette section présente la partie du projet consacrée au profiling comportemental des joueurs. L’objectif est de compléter l’estimation du niveau Elo par une description plus qualitative de la manière de jouer : rythme, prise de risque, stabilité, gestion du temps et fragilité selon les phases de jeu. L’idée centrale est de comparer chaque joueur à des joueurs de niveau similaire, afin d’identifier ce qui le distingue réellement. Les paragraphes suivants décrivent l’approche retenue, l’attribution des profils sémantiques, puis les résultats obtenus avec 30 et 10 parties par joueur.

5.1 Approche générale du profiling comportemental

Les profils sont construits à partir des corpus bullet, blitz et rapid déjà décrits dans les chapitres précédents. Deux configurations sont étudiées : une version avec 30 parties par joueur (*lowN30*) et une version avec 10 parties par joueur (*lowN10*), afin de tester la robustesse du profiling lorsque peu de parties sont disponibles.

Les variables utilisées sont les features comportementales calculées au niveau joueur-cadence. Elles couvrent principalement la qualité des coups, le temps de réflexion, les erreurs sous pression, la performance par phase de jeu, la stabilité inter-parties et la conversion des positions favorables. Ces familles de variables sont ensuite résumées en dimensions de style plus lisibles.

TABLE 5.1 – Familles de features utilisées pour le profiling comportemental

Famille	Exemples de variables	Rôle dans le profil
Temps et rythme	<code>time_mean</code> , <code>time_p90</code> , <code>good_fast_rate</code>	Identifier les joueurs rapides ou lents relativement à leur niveau.
Pression temporelle	<code>low_clock_rate</code> , <code>blunder_low_clock</code>	Mesurer la fragilité lorsque le temps restant devient faible.
Activité tactique	<code>rate_capture</code> , <code>rate_check</code> , <code>critical_rate</code>	Décrire le caractère tactique ou positionnel du jeu.
Risque et erreurs	<code>rate_blunder</code> , <code>rate_mistake</code> , <code>loss_p90</code>	Détecter les pertes importantes et la prise de risque.
Phases de jeu	<code>loss_opening</code> , <code>loss_endgame</code>	Repérer les forces et fragilités selon les moments de la partie.
Stabilité et conversion	<code>loss_std</code> , <code>loss_when_winning</code>	Mesurer la régularité et la capacité à convertir un avantage.

5.1.1 Construction des scores de style

La première étape consiste à transformer les features brutes en scores comparables. Comme les variables n'ont pas la même unité, une normalisation robuste est appliquée :

$$z_{i,j} = \frac{x_{i,j} - \text{median}(x_j)}{\text{IQR}(x_j)},$$

où $x_{i,j}$ désigne la valeur de la variable j pour le joueur i . L'utilisation de l'IQR rend la normalisation moins sensible aux valeurs extrêmes, fréquentes dans les pertes en centipions ou les temps de réflexion.

Les variables normalisées sont ensuite combinées en dimensions de style. Chaque dimension regroupe plusieurs features cohérentes et attribue un signe à chaque variable selon son interprétation. De manière générale, une dimension k est calculée sous la forme :

$$d_{i,k} = \frac{1}{|F_k|} \sum_{j \in F_k} \alpha_{j,k} z_{i,j},$$

où F_k est l'ensemble des features associées à la dimension k , et $\alpha_{j,k}$ vaut $+1$ ou -1 selon le sens attendu de la variable. Par exemple, un temps de réflexion plus faible augmente le score *rapide*, tandis qu'un taux de blunders plus élevé augmente le score *risqué*.

TABLE 5.2 – Dimensions finales de style

Dimension	Score positif	Score négatif	Interprétation
Rythme de jeu	Rapide	Lent	Vitesse de jeu relativement au niveau.
Agressivité tactique	Agressif	Calme / positionnel	Recherche de positions tactiques ou critiques.
Risque d'erreurs	Risqué	Propre	Propension aux erreurs importantes.
Pression temporelle	Sous pression temps	À l'aise au temps	Dégradation lorsque le temps est faible.
Finale	Fragile en finale	Stable en finale	Qualité relative dans la phase finale.
Stabilité	Instable	Régulier	Variabilité des performances entre parties.
Conversion	Convertit mal	Bon convertisseur	Capacité à conserver une position favorable.

5.1.2 Attribution des profils sémantiques

Une fois les dimensions de style calculées, l'objectif est d'attribuer un profil lisible à chaque joueur. Le Elo n'est pas utilisé comme définition du style : il sert uniquement de contexte de comparaison. Un joueur est donc comparé à des joueurs de niveau proche, dans la même cadence.

Pour éviter qu'un profil comme *propre* corresponde simplement aux joueurs les plus forts, chaque dimension est d'abord corrigée par rapport au Elo prédit :

$$r_{i,k} = d_{i,k} - \widehat{d}_k(\widehat{\text{Elo}}_i),$$

où $d_{i,k}$ est le score observé et $\widehat{d}_k(\widehat{\text{Elo}}_i)$ le score attendu pour un joueur de niveau similaire. Ensuite, une comparaison locale est appliquée à partir des voisins les plus proches en Elo prédit :

$$s_{i,k} = \frac{r_{i,k} - \text{median}(r_{\mathcal{N}_{i,k}})}{\text{IQR}(r_{\mathcal{N}_{i,k}})}.$$

Le score final $s_{i,k}$ indique donc si le joueur est rapide, risqué, instable ou fragile en finale par rapport à des joueurs comparables.

Le nom du profil est construit en conservant les dimensions les plus marquées. Si plusieurs scores dépassent le seuil retenu, les traits sont combinés, par exemple *rapide + agressif + instable* ou *lent + propre + sous pression temps*. Les combinaisons trop rares sont simplifiées vers les traits dominants, et les joueurs sans trait suffisamment net sont placés dans une catégorie mixte.

5.1.3 Protocole d'évaluation

La qualité des profils est évaluée avec quatre critères. L'**interprétabilité** vérifie que le nom du profil correspond bien aux scores internes. L'**alignement** mesure la cohérence entre les traits attribués et les dimensions de style. La **stabilité** teste si les profils restent proches lorsque les scores sont légèrement perturbés. Enfin, la **cohésion** compare la distance moyenne entre joueurs d'un même profil à la distance moyenne globale ; un ratio inférieur à 1 indique que les joueurs d'un même profil sont plus proches entre eux que des joueurs pris au hasard.

Un dernier indicateur, η^2 , mesure la part de variance du Elo expliquée par les profils. Une valeur faible est souhaitable ici, car le but n'est pas de reconstruire des tranches de niveau, mais d'obtenir une information comportementale complémentaire au Elo.

5.2 Résultats et discussion

5.2.1 Résultats avec 30 parties par joueur

Le protocole *lowN30* utilise 30 parties par joueur. Les résultats montrent que les profils obtenus sont très majoritairement interprétables, cohérents et faiblement liés au Elo.

TABLE 5.3 – Qualité des profils sémantiques avec 30 parties par joueur

Cadence	Profils	Interp. (%)	Alignement	Stabilité	Cohésion	η^2 Elo
Bullet	23	98.3	0.939	0.718	0.637	0.038
Blitz	22	97.7	0.931	0.720	0.648	0.043
Rapid	19	97.7	0.953	0.756	0.651	0.051

Les ratios de cohésion sont tous inférieurs à 1. Les joueurs appartenant à un même profil sont donc plus proches entre eux que des joueurs pris au hasard. Les valeurs de η^2 restent faibles, entre 0.038 et 0.051, ce qui confirme que les profils ne représentent pas simplement des tranches de niveau.

TABLE 5.4 – Exemples de profils fréquents avec 30 parties

Cadence	Profils fréquents
Bullet	fragile en finale, convertit mal, sous pression temps, rapide, stable en finale, propre, agressif, bon convertisseur, lent, risqué, instable.
Blitz	lent, fragile en finale, convertit mal, sous pression temps, régulier, agressif, calme/positionnel, instable, bon convertisseur, propre.
Rapid	sous pression temps, agressif, fragile en finale, lent, convertit mal, régulier, calme/positionnel, rapide, instable.

Ces profils montrent que le style n’est pas réduit à un unique axe. On observe des profils liés au temps, à la qualité des coups, à la stabilité, mais aussi aux phases de jeu. Cette diversité confirme l’intérêt d’une représentation multidimensionnelle du joueur.

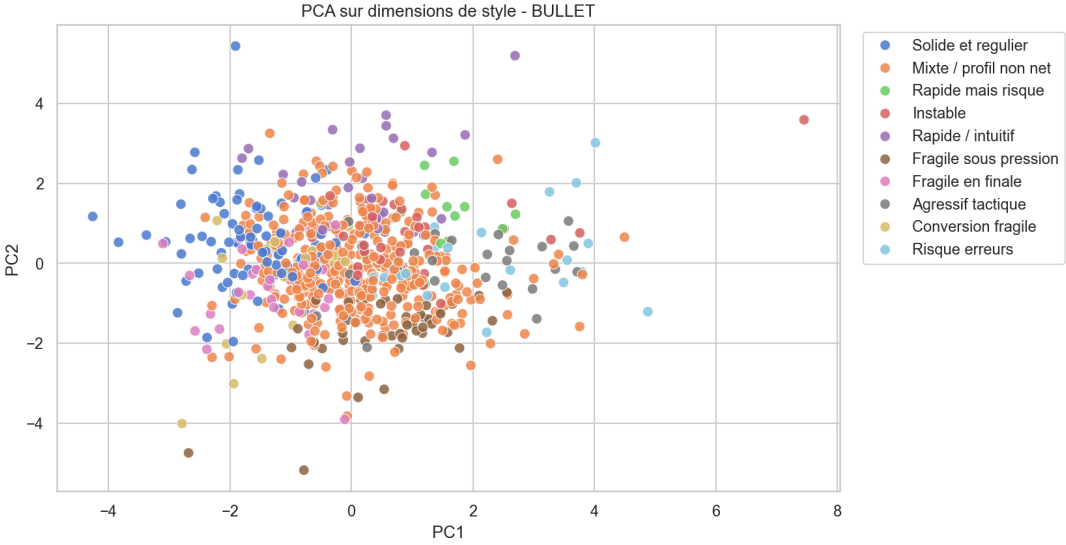


FIGURE 5.1 – Projection PCA des profils sémantiques en bullet avec 30 parties par joueur

La projection PCA montre un chevauchement partiel des profils. Les profils ne correspondent donc pas à des classes naturellement séparées, mais plutôt à des zones de l’espace comportemental construites à partir de combinaisons de traits continus. Cette visualisation confirme l’intérêt d’une typologie sémantique interprétable plutôt qu’un clustering strict.

5.3 Résultats avec 10 parties par joueur

Un second protocole utilise seulement 10 parties par joueur. L’objectif est de vérifier si la méthode reste exploitable dans un contexte où peu de parties sont disponibles. Ce cas est important pour un futur moteur adaptatif, car un nouvel utilisateur ne disposera pas toujours d’un grand historique de parties.

TABLE 5.5 – Qualité des profils sémantiques avec 10 parties par joueur

Cadence	Profils	Interp. (%)	Alignement	Stabilité	Cohésion	η^2 Elo
Bullet	23	97.7	0.942	0.711	0.637	0.031
Blitz	21	97.9	0.940	0.734	0.641	0.040
Rapid	16	98.8	0.931	0.793	0.651	0.022

Les résultats restent proches de ceux obtenus avec 30 parties. L’interprétabilité reste très élevée, la cohésion demeure inférieure à 1 et le lien avec le Elo reste faible.

La stabilité est même légèrement meilleure en rapid dans ce protocole, même si cette observation doit être interprétée avec prudence en raison du nombre plus faible de joueurs dans cette cadence.

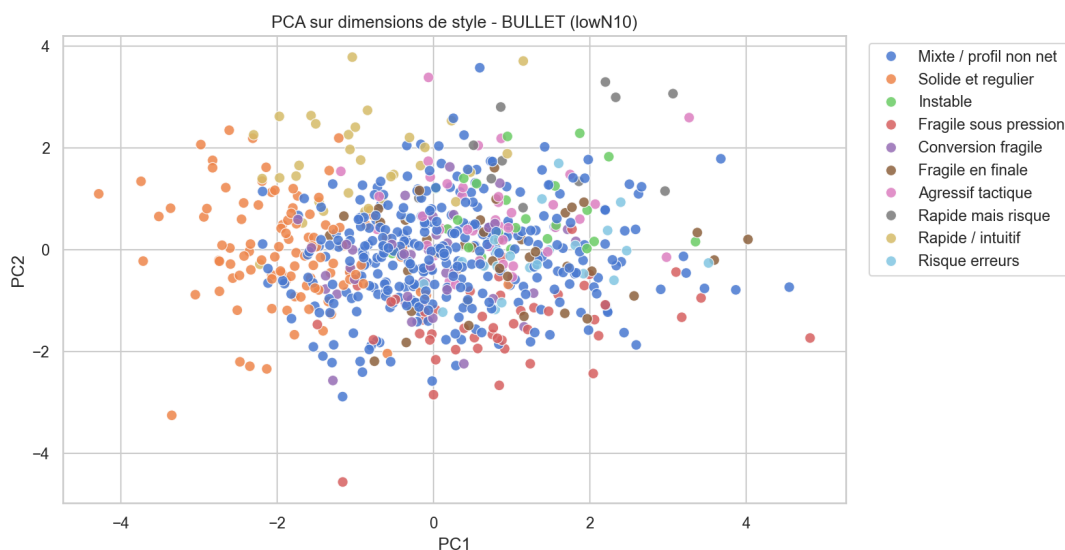


FIGURE 5.2 – Projection PCA des profils sémantiques en bullet avec 10 parties par joueur

La projection PCA avec 10 parties conserve la même lecture générale que celle du protocole à 30 parties. Le nuage de points reste continu et les profils se recouvrent partiellement, ce qui suggère que la réduction du nombre de parties ajoute du bruit sans transformer les profils en classes strictement séparées.

TABLE 5.6 – Performance du modèle Elo utilisé comme contexte

Cadence	30 parties			10 parties		
	MAE	RMSE	R^2	MAE	RMSE	R^2
Bullet	148.0	186.5	0.649	174.4	216.3	0.528
Blitz	140.8	178.6	0.640	163.4	205.7	0.523
Rapid	143.2	177.2	0.596	163.7	205.3	0.458

La baisse de performance du modèle Elo avec 10 parties est attendue : moins de parties signifie moins d'information sur le joueur. Cependant, les profils sémantiques restent lisibles et cohérents, car ils s'appuient sur des dimensions comportementales normalisées localement.

5.4 Discussion

Les résultats indiquent que le profiling sémantique est plus adapté que le clustering classique pour décrire les joueurs. Les silhouettes faibles obtenues par clustering montrent que les profils ne forment pas des classes naturelles très séparées. Cependant, les scores de style permettent de produire des descriptions lisibles et utiles.

La principale conclusion est que le Elo et le profil ne mesurent pas la même chose. Le Elo résume le niveau global du joueur, tandis que le profil décrit sa manière de jouer relativement à des joueurs de niveau similaire. Les faibles valeurs de η^2 confirment que les profils n'expliquent qu'une petite part de la variance du Elo. Ils apportent donc une information complémentaire.

Cette distinction est importante pour le projet PMind. Dans un moteur adaptatif, le Elo peut servir à régler la force générale de l'opposition, tandis que le profil peut orienter le comportement ou les exercices proposés. Par exemple, un joueur *sous pression temps* pourrait recevoir des positions d'entraînement avec contrainte temporelle, tandis qu'un joueur *fragile en finale* pourrait être orienté vers des exercices de conversion technique.

5.5 Limites

Plusieurs limites doivent être mentionnées. Premièrement, les profils sémantiques ne doivent pas être interprétés comme des classes naturelles parfaitement séparées. Ils constituent une typologie interprétable fondée sur des scores continus.

Deuxièmement, les combinaisons rares sont moins stables que les profils simples. Les profils comme *rapide* ou *fragile en finale* sont plus robustes que des combinaisons très spécifiques comportant plusieurs traits.

Troisièmement, la correction par Elo dépend de la qualité du modèle Elo utilisé comme contexte. Lorsque seulement 10 parties sont disponibles, l'estimation du Elo est moins précise, ce qui peut introduire davantage de bruit dans les comparaisons locales.

Enfin, les dimensions de style restent liées aux features construites. D'autres indicateurs, par exemple des représentations plus fines de la structure de position ou des motifs tactiques, pourraient enrichir les profils dans une version future.

5.6 Conclusion

Cette partie montre qu'il est possible de construire un profil comportemental interprétable à partir des parties d'un joueur. La méthode retenue repose sur des scores de style corrigés par le Elo prédit, puis comparés localement à des joueurs de niveau

similaire. Les profils obtenus sont lisibles, relativement stables, cohérents et faiblement liés au Elo.

Le résultat principal peut être résumé ainsi :

Le Elo indique le niveau du joueur, tandis que le profil sémantique décrit sa manière de jouer.

Cette séparation entre niveau et style constitue une base pertinente pour un moteur d'échecs adaptatif. Elle permet non seulement d'ajuster la force du moteur, mais aussi d'identifier les comportements typiques du joueur afin de proposer une expérience d'entraînement plus personnalisée.

Conclusion générale

Bilan du travail

Ce mémoire avait pour objectif d'étudier la possibilité de construire les bases d'un moteur d'échecs adaptatif, capable de ne pas se limiter à une estimation globale du niveau d'un joueur, mais aussi de décrire sa manière de jouer. La question centrale était donc double : peut-on estimer le Elo d'un joueur à partir d'un nombre limité de parties, et peut-on compléter cette estimation par un profil comportemental interprétable ? Les travaux menés montrent que ces deux dimensions sont complémentaires. Le Elo donne une indication de force générale, tandis que le profil comportemental permet de caractériser des tendances plus fines : gestion du temps, stabilité, agressivité, qualité des coups ou fragilité selon les phases de jeu.

La première contribution de ce travail concerne la construction du corpus de données. Les parties ont été collectées à partir de Lichess, en distinguant les cadences bullet, blitz et rapid, puis organisées selon trois niveaux d'analyse : le coup, la partie et le joueur. Cette structuration a permis de transformer des données brutes de parties en un dataset exploitable pour l'apprentissage automatique. Les évaluations moteur, les informations temporelles, les résultats, les ouvertures et les phases de jeu ont été utilisées pour construire des variables agrégées décrivant le comportement des joueurs. Cette étape de feature engineering constitue une base importante du projet, car elle transforme les traces de jeu en indicateurs interprétables.

Principaux résultats

L'analyse exploratoire a montré que les cadences ne peuvent pas être traitées comme un ensemble homogène. Le bullet, le blitz et le rapid présentent des distributions différentes en termes de durée, de nombre de coups, de pertes en centipions et de fréquence des erreurs. Cette observation justifie la séparation des expériences par cadence. L'étude des corrélations a également mis en évidence que certaines variables sont liées au niveau Elo, tandis que d'autres décrivent davantage des comportements de jeu. Cette distinction est essentielle pour le reste du mémoire : toutes les variables comportementales ne mesurent pas directement la force d'un joueur, et certaines peuvent servir à décrire

son style.

La partie modélisation a confirmé qu'il est possible d'estimer le niveau d'un joueur à partir d'un nombre réduit de parties, même si la précision diminue lorsque l'information disponible devient plus limitée. Les expériences ont permis de comparer plusieurs stratégies : modèles globaux, séparation par cadence, sélection de features, ajout d'embeddings et prédiction au niveau partie suivie d'une agrégation. Les meilleurs résultats montrent que les modèles supervisés, notamment les approches de gradient boosting, exploitent efficacement les variables comportementales construites. Les performances obtenues avec seulement dix parties restent naturellement moins stables qu'avec davantage de données, mais elles demeurent suffisamment informatives pour envisager une première estimation rapide du niveau d'un nouvel utilisateur.

La dernière contribution du mémoire porte sur la construction de profils comportementaux sémantiques. L'idée principale a été de ne pas confondre niveau et style : deux joueurs de même Elo peuvent avoir des façons de jouer très différentes. Pour cette raison, les scores de style ont été corrigés par rapport au Elo prédit, puis comparés localement à des joueurs de niveau similaire. Les profils obtenus, tels que *sous pression temps*, *fragile en finale*, *convertit mal*, *rapide*, *propre* ou *agressif*, fournissent une description lisible du joueur. Les résultats montrent une forte interprétabilité, une cohésion inférieure à 1 et un faible lien direct avec le Elo, ce qui confirme que ces profils apportent une information complémentaire au niveau.

Synthèse générale

Le résultat principal de ce travail peut être résumé ainsi :

Le Elo estime le niveau du joueur, tandis que le profil comportemental décrit sa manière de jouer.

Cette séparation constitue un point important pour la conception d'un moteur adaptatif. Un moteur personnalisé ne doit pas seulement ajuster sa force ; il doit aussi pouvoir adapter son comportement ou ses recommandations aux caractéristiques du joueur. Par exemple, un joueur *fragile en finale* pourrait recevoir davantage d'exercices de conversion technique, tandis qu'un joueur souvent *sous pression temps* pourrait bénéficier de positions d'entraînement avec contrainte temporelle.

Limites

Ce travail comporte néanmoins plusieurs limites. Le corpus rapid est plus réduit que les corpus bullet et blitz, ce qui peut limiter la généralisation des résultats pour cette cadence. Les features restent également dépendantes des choix de construction

effectués : elles décrivent certaines dimensions du style, mais ne capturent pas toute la richesse stratégique et tactique d'une partie d'échecs.

De plus, l'utilisation de seulement dix parties introduit du bruit dans l'estimation du niveau et du profil, car une courte séquence de parties peut refléter une période atypique du joueur. Enfin, les profils sémantiques ne doivent pas être interprétés comme des classes naturelles parfaitement séparées : ils correspondent plutôt à des zones dans un espace continu de comportements.

Perspectives

Plusieurs perspectives peuvent prolonger ce travail. Une première piste consisterait à enrichir les représentations utilisées, par exemple avec des embeddings de positions, des motifs tactiques récurrents ou des indicateurs plus fins de structure de pions, d'activité des pièces et de sécurité du roi. Une deuxième piste serait de mettre à jour le profil du joueur en temps réel, au fil des nouvelles parties, afin de suivre son évolution.

Enfin, l'intégration de ces profils dans un moteur d'échecs adaptatif ouvrirait des applications concrètes : génération d'exercices ciblés, choix de positions adaptées, simulation d'adversaires personnalisés et accompagnement pédagogique plus individualisé.

Ouverture

En conclusion, ce mémoire montre que l'apprentissage automatique appliqué aux échecs ne permet pas seulement de prédire un niveau. Il peut aussi contribuer à modéliser une identité de jeu, c'est-à-dire une manière de prendre des décisions sur l'échiquier. Cette dimension comportementale est essentielle pour dépasser les moteurs simplement réglés par force et aller vers des systèmes réellement personnalisés, capables d'accompagner chaque joueur selon son niveau, ses habitudes et ses axes de progression.

Bibliographie

- [1] FIDE. *Laws of Chess*. 2023. Disponible en ligne : <https://www.fide.com/FIDE/handbook/LawsOfChess.pdf>
- [2] Elo, A. E. *The Rating of Chessplayers, Past and Present*. Arco Publishing, 1978.
- [3] Encyclopaedia Britannica. *Chess*. Disponible en ligne : <https://www.britannica.com/topic/chess>
- [4] McIlroy-Young, R., Sen, S., Kleinberg, J., Anderson, A. *Aligning Superhuman AI with Human Behavior : Chess as a Model System*. Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020. Disponible en ligne : <https://doi.org/10.1145/3394486.3403219>
- [5] Silver, D. et al. *A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play*. Science, 362(6419), 1140–1144, 2018. Disponible en ligne : <https://www.science.org/doi/10.1126/science.aar6404>
- [6] Stockfish Chess Engine. *Official website*. Disponible en ligne : <https://stockfishchess.org/>
- [7] Leela Chess Zero. *Overview documentation*. Disponible en ligne : <https://lczero.org/dev/overview/>
- [8] Chess Programming Wiki. *Search*. Disponible en ligne : <https://www.chessprogramming.org/Search>
- [9] Chess Programming Wiki. *Evaluation*. Disponible en ligne : <https://www.chessprogramming.org/Evaluation>
- [10] Stockfish Documentation. *Stockfish FAQ — centipawn / normalized evaluation*. Disponible en ligne : <https://official-stockfish.github.io/docs/stockfish-wiki/Stockfish-FAQ.html>
- [11] Chess Programming Wiki. *NNUE*. Disponible en ligne : <https://www.chessprogramming.org/NNUE>
- [12] Stockfish Blog. *Introducing NNUE Evaluation*. 2020. Disponible en ligne : <https://stockfishchess.org/blog/2020/introducing-nnue-evaluation/>
- [13] McIlroy-Young, R., Wang, R., Sen, S., Kleinberg, J., Anderson, A. *Learning Personalized Models of Human Behavior in Chess*. International Conference on Machine

- Learning, 2021. Disponible en ligne : <https://proceedings.mlr.press/v139/mcilroy-young21a.html>
- [14] McIlroy-Young, R., Wang, R., Sen, S., Kleinberg, J., Anderson, A. *Detecting Individual Decision-Making Style : Exploring Behavioral Stylometry in Chess*. NeurIPS, 2021 / arXiv, 2022. Disponible en ligne : <https://arxiv.org/abs/2112.02547>
- [15] Tang, Z., Jiao, D., Xue, E., McIlroy-Young, R., Kleinberg, J., Sen, S., Anderson, A. *Learning to Imitate with Less : Efficient Individual Behavior Modeling in Chess*. arXiv, 2025. Disponible en ligne : <https://arxiv.org/abs/2502.17316>
- [16] Lichess. *Plateforme d'échecs en ligne*. Disponible en ligne : <https://lichess.org/fr>
- [17] Newborn, M. *Kasparov versus Deep Blue : Computer Chess Comes of Age*. Springer, 1997